

Lecture9a

October 31, 2025

1 CS5489 - Machine Learning

2 Lecture 9a - Unsupervised Learning - Clustering

2.0.1 Dept. of Computer Science, City University of Hong Kong

3 Outline

1. Unsupervised Learning
2. Parametric clustering
 1. K-means
 2. Gaussian mixture models (GMMs)
 3. Bayesian GMMs
3. Non-parametric clustering and Mean-shift
4. Spectral clustering

```
[1]: # setup
%matplotlib inline
import matplotlib_inline # setup output image format
matplotlib_inline.backend_inline.set_matplotlib_formats('retina')
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 100 # display larger images
import matplotlib
from numpy import *
from sklearn import *
from scipy import stats
from mpl_toolkits.mplot3d import Axes3D
import skimage.util
```

4 Unsupervised Learning

- Unsupervised learning only considers the input data $\mathbf{x}$.
 - There are no output values.
- **Goal:** Try to discover inherent properties in the data.
 - Clustering
 - Dimensionality Reduction
 - Manifold Embedding

5 Clustering

- Find clusters of similar items in the data.
 - Find a representative item that can represent all items in the cluster.
- **For example:** grouping iris flowers by their measurements.
 - Features are sepal width and petal length.

6 Clustering

- Data is set of vectors $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$
 - Each data point is a vector $\mathbf{x} \in \mathbb{R}^d$.
- **Goal:** group similar data together.
 - groups are also called clusters.
 - each data point is assigned with a cluster index ($y \in \{1, \dots, K\}$)
 - * K is the number of clusters.

```
[2]: axbox = [0, 8, 1.5, 5]

# make a colormap for viewing classes
mycmap = matplotlib.colors.ListedColormap(["#FF0000", "#00FF00", "#0000FF"])
cols = ['r', 'g', 'b']
jet = plt.get_cmap('jet')
rbow = plt.get_cmap('rainbow')

al = 0.4
rbow_data2 = {
    'red': lambda x: (1-al) + al*abs(2 * x - 0.5),
    'green': lambda x: (1-al) + al*sin(x * pi),
    'blue': lambda x: (1-al) + al*cos(x * pi / 2),
}
rbow2 = matplotlib.colors.LinearSegmentedColormap('rainbow2', rbow_data2)

# load iris data each row is (petal length, sepal width, class)
irisdata = loadtxt('iris3.csv', delimiter=',', skiprows=1)

X = irisdata[:,0:2] # the first two columns are features (petal length, sepal
↪width)
Y = irisdata[:,2] # the third column is the class label (setosa=0,
↪versicolor=1, virginica=2)

km = cluster.KMeans(n_clusters=3, random_state=4487)
Yp = km.fit_predict(X)

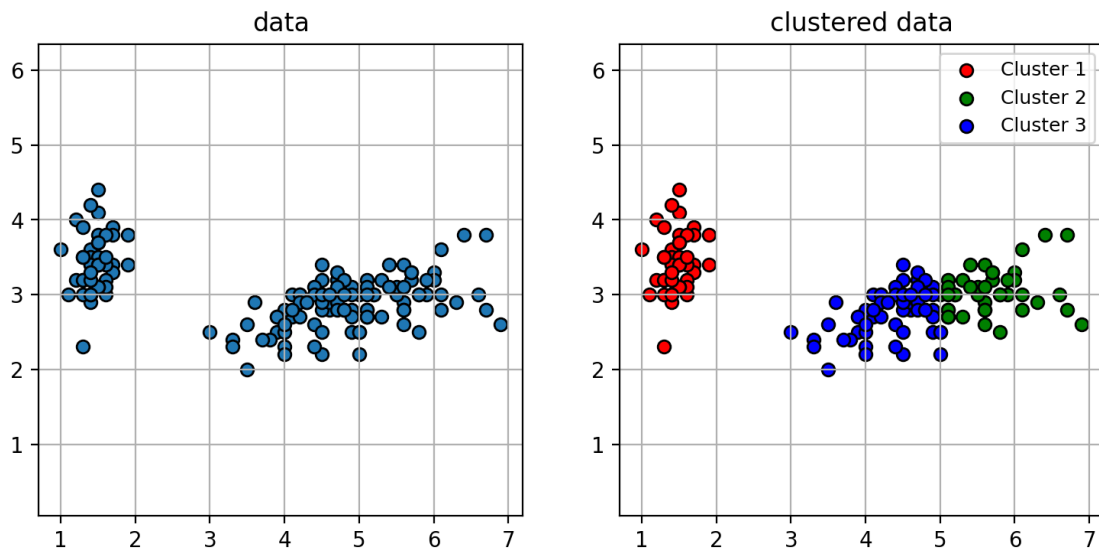
clusterfig = plt.figure(figsize=(9,4))
plt.subplot(1,2,1)
plt.scatter(X[:,0], X[:,1], edgecolors='k')
plt.axis(axbox); plt.axis('equal'); plt.grid(True)
```

```
plt.title('data')

plt.subplot(1,2,2)
for j in range(3):
    inds = (Yp == j)
    plt.scatter(X[inds,0], X[inds,1], c=cols[j], label='Cluster ' +str(j+1),
    ↪edgecolors='k')
plt.axis(axbox); plt.axis('equal'); plt.grid(True)
plt.title('clustered data')
leg = plt.legend(fontsize=9)
plt.close()
```

[3]: clusterfig

[3]:



7 K-Means Clustering

- **Idea:**

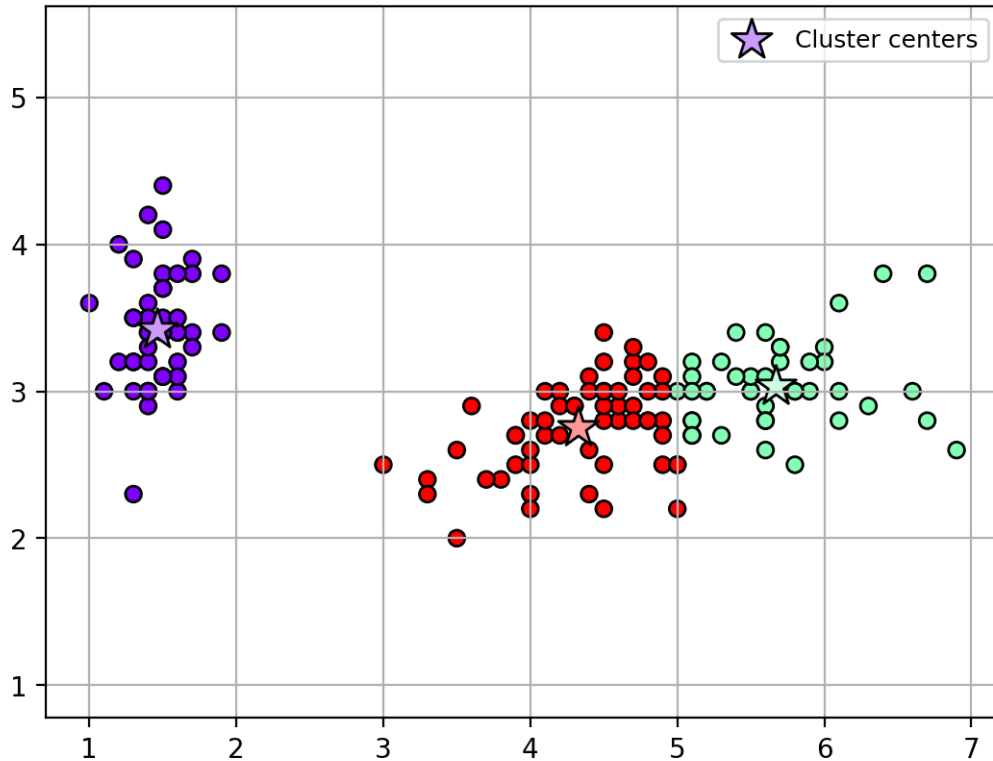
- there are K clusters.
- each cluster is represented by a *cluster center*.
 - * $\mathbf{c}_j \in \mathbb{R}^d$, $j \in \{1, \dots, K\}$
- assign each data point to the closest cluster center.
 - * assignment variable $z_i \in \{1, \dots, K\}$ indexes the cluster center of $\mathbf{x}_i$.

```
[4]: cc = km.cluster_centers_
kmfig = plt.figure()
plt.scatter(X[:,0], X[:,1], c=Yp, cmap=rbow, edgecolors='k')
plt.scatter(cc[:,0], cc[:,1], c=arange(3), s=250, cmap=rbow2, marker='*',
    ↪label='Cluster centers', edgecolors='k')
```

```
plt.axis(axbox); plt.axis('equal'); plt.grid(True)
plt.legend(fontsize=9)
plt.close()
```

[5]: kmfig

[5]:



8 K-means Clustering Objective

- **Objective:** minimize the total sum-squared difference between points and their centers

$$\min_{\mathbf{c}_1, \dots, \mathbf{c}_K, z_1, \dots, z_n} \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{c}_{z_i}\|^2$$

9 K-means Clustering Objective

- *How to pick the cluster centers?*
 - Assume the assignments z_i are known.
 - Pick the cluster centers that minimize the squared distance to all its cluster members.

$$\min_{\mathbf{c}_1, \dots, \mathbf{c}_K} \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{c}_{z_i}\|^2$$

- Solution:

- if the assignments $\{z_i\}$ are known...
 - * let C_j be the set of points assigned to cluster j
 - $C_j = \{\mathbf{x}_i | z_i = j\}$
 - * For each cluster, we have
 - $\mathbf{c}_j = \operatorname{argmin} \sum_{i \in C_j} \|\mathbf{x}_i - \mathbf{c}_j\|^2$
 - * Take the derivative and set to 0:

$$\begin{aligned} \frac{d}{d\mathbf{c}_j} \sum_{i \in C_j} \|\mathbf{x}_i - \mathbf{c}_j\|^2 &= \sum_{i \in C_j} 2(\mathbf{x}_i - \mathbf{c}_j)(-1) = 0 \\ \Rightarrow \sum_{i \in C_j} \mathbf{x}_i - |C_j|\mathbf{c}_j &= 0 \end{aligned}$$

- * Cluster center is the mean of the points in the cluster
 - $\mathbf{c}_j = \frac{1}{|C_j|} \sum_{\mathbf{x}_i \in C_j} \mathbf{x}_i$

10 K-means Clustering Objective

- *How to pick the assignments?*
 - Assume the clusters $\{\mathbf{c}_1, \dots, \mathbf{c}_K\}$ are *known*.
 - Pick the assignments that minimize the squared distance to the clusters.

$$\min_{z_1, \dots, z_n} \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{c}_{z_i}\|^2$$

- For each data point,
 - * $z_i = \operatorname{argmin}_{j \in \{1, \dots, K\}} \|\mathbf{x}_i - \mathbf{c}_j\|^2$
 - * i.e., assign point $\mathbf{x}_i$ to its closest cluster.

11 Chicken and Egg Problem

- Cluster assignment of each point depends on the cluster centers.
- Location of cluster center depends on which points are assigned to it.
- **Solution:** just iterate between the two steps.
 - Note: in each step we are holding one set of variables fixed, while minimizing over the others.
 - Thus we are always minimizing the original objective!

12 K-means Algorithm

- Pick initial cluster centers
- Repeat:
 - 1) calculate assignment z_i for each point $\mathbf{x}_i$: closest cluster center using Euclidean distance.
 - 2) calculate cluster center $\mathbf{c}_j$ as average of points assigned to cluster j .
- This procedure will converge eventually.

```
[30]: # ax is the axis
      # cmap is colormap to use
```

```

# cl is the index of components to show [default is None which means all
↳ components]
def make_ellipses(means, covars, ax, cmap, cl=None, showcvlines=False):
    dim = means.shape[1]
    if cl is None:
        cl = range(means.shape[0])
    for n in cl:
        mycv = covars[n]
        if mycv.size==1:
            mycv = mycv*eye(dim)
        elif mycv.size==dim:
            mycv = diag(mycv)

        v, w = linalg.eigh(mycv)

        u = w[0] / linalg.norm(w[0])
        angle = arctan2(u[1], u[0])
        angle = 180 * angle / pi # convert to degrees
        s = sqrt(v) # stddev

        for f in [1,2,3]:
            ss = s*f
            if not showcvlines:
                # filled patch
                ell = matplotlib.patches.Ellipse(means[n, :2], 2*ss[0], 2*ss[1],
                                                    angle=180 + angle,
↳ color=cmap(float(n)/(lambda a,b:a if a>b else b)(max(cl),1)))
                ell.set_clip_box(ax.bbox)
                ell.set_alpha(0.15)
                ax.add_artist(ell)
            else:
                # line patch
                ell = matplotlib.patches.Ellipse(means[n, :2], 2*ss[0], 2*ss[1],
                                                    angle=180 + angle,
↳ color=cmap(float(n)/(lambda a,b:a if a>b else b)(max(cl),1)), fill=False)
                ell.set_clip_box(ax.bbox)
                ell.set_alpha(0.5)
                ax.add_artist(ell)

def plot_clusters(model, axbox, X, Y, pcmap, ccmap, showregions=False,
↳ showprob=False, showcvlines=False):
    # model = kmeans, gmm, dpvgmm
    # X = points
    # pcmap = colormap for points
    # ccmap = colormap for centers

```

```

name = model.__class__.__name__

if (name == 'KMeans'):
    cc = model.cluster_centers_
    cl = range(model.n_clusters)
    cv = None
elif (name == 'GMM') or (name == 'GaussianMixture'):
    cc = model.means_
    cl = range(model.n_components)
    cv = model.covariances_
elif (name == 'DPGMM') or (name == 'BayesianGaussianMixture'):
    cl = unique(Y)          # find active clusters
    cc = model.means_[cl]   # get means
    cv = model.covariances_[cl]
elif (name == 'VBGMM'):
    cl = unique(Y)          # find active clusters
    cc = model.means_[cl]   # get means
    cv = None
elif (name == 'MeanShift'):
    cc = model.cluster_centers_
    cl = range(len(cc))
    cv = None
elif (name == 'SpectralClustering'):
    cc = None
    cl = range(model.n_clusters)
    cv = None

if showregions:
    dx = axbox[1]-axbox[0]
    dy = axbox[3]-axbox[2]

    axbox2 = [ axbox[0]-0.5*dx, axbox[1]+0.5*dx,
               axbox[2]-0.5*dy, axbox[3]+0.5*dy]

    # grid of points
    xr = [linspace(axbox2[0],axbox2[1],300),
          linspace(axbox2[2],axbox2[3],300)]
    xgrid0, xgrid1 = meshgrid(xr[0], xr[1])
    allpts = c_[xgrid0.ravel(), xgrid1.ravel()]

    # predictions
    Ypred = model.predict(allpts).reshape(xgrid0.shape)

    plt.imshow(Ypred, origin='lower', extent=axbox2, aspect='auto',
               vmin=min(cl), vmax=max(cl), alpha=0.2, cmap=pcmap)

# show ellipses

```

```

if (cv is not None) and (not showregions):
    make_ellipses(cc, cv, plt.gca(), pcmap, None, showcvlines)

# show points
if X is not None:
    if not showprob:
        # show class labels
        plt.scatter(X[:,0], X[:,1], c=Y, cmap=pcmap, edgecolors='k')
    else:
        # get colors for each class
        K = max(Y)+1
        Kcols = array([pcmap(i*1./(K-1)) for i in range(K)]) # K x C
        # get probabilities for each class
        Yp = model.predict_proba(X) # N x K
        # build the colors for each sample
        Ycols = Yp @ Kcols[:,0:3]
        plt.scatter(X[:,0], X[:,1], c=Ycols, edgecolors='k')

# show centers
if cc is not None:
    plt.scatter(cc[:,0], cc[:,1], c=c1, s=250, cmap=ccmap, marker='*',
    ↪edgecolors='k')

plt.axis('equal'); plt.axis(axbox); plt.grid(True)

# return number of clusters found
return (len(c1),)

```

```

[31]: random.seed(123)
indinit = random.permutation(len(X))[0:3]
ccinit = X[indinit,:]

for i in range(6):
    if i==0:
        kmifig1 = plt.figure(figsize=(10,3))
        py = 1
        px = 2
        ioff = 0
        tstr = 'initial'
        km = cluster.KMeans(n_clusters=3, max_iter=1, n_init=1, init=ccinit)
        Yp = km.fit_predict(X)
        km.cluster_centers_ = ccinit # force clusters to initial points (hack)
        ↪
        Yp = km.predict(X)
    else:

```

```

km = cluster.KMeans(n_clusters=3, max_iter=i, n_init=1, init=ccinit)
Yp = km.fit_predict(X)
tstr = 'iteration ' + str(i)

plt.subplot(py,px,i+1+ioff)
plot_clusters(km, axbox, X, Yp, rbow, rbow2)
plt.title(tstr)

if i==1:
    plt.close()
    kmifig2 = plt.figure(figsize=(10,6))
    py = 2
    px = 2
    ioff = -2
plt.close()

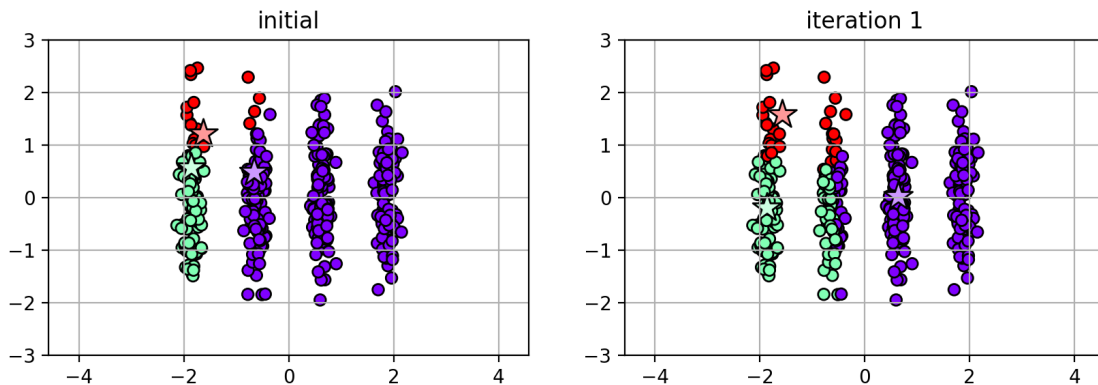
```

[32]: kmifig1

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

[32]:



[33]: kmifig2

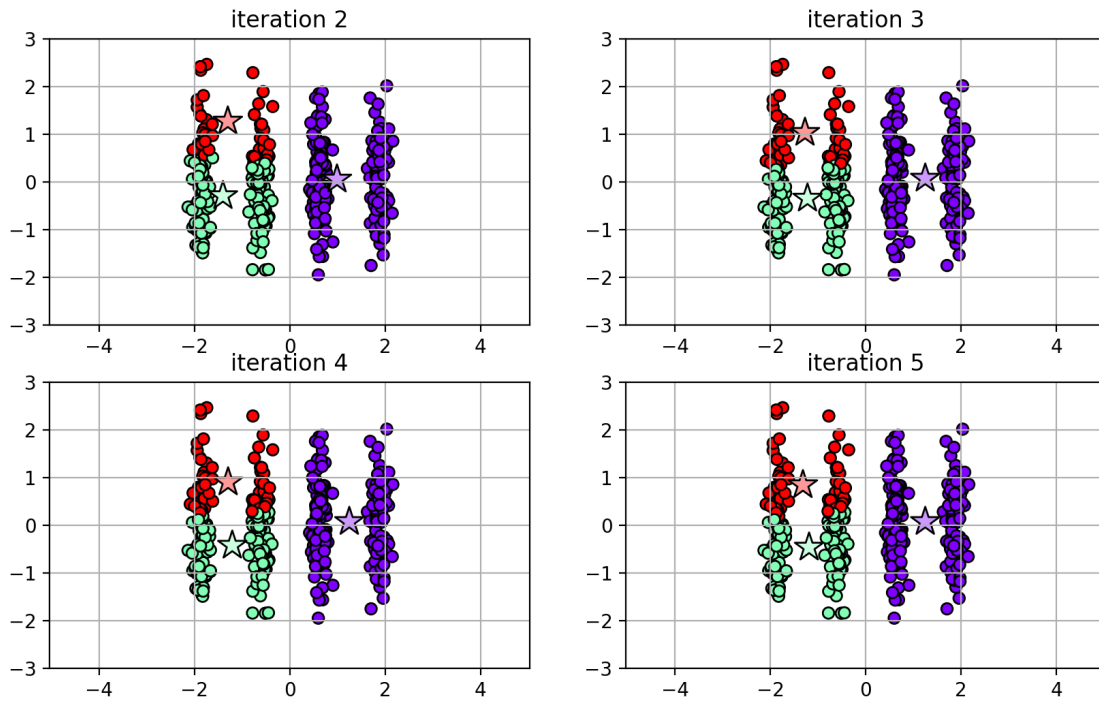
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

[33]:



13 Important Note

- The final result depends on the initial cluster centers!
 - Some bad initializations will yield poor clustering results!
 - (Technically, there are multiple local minimums in the objective function)

```
[34]: random.seed(126)
indinit = random.permutation(len(X))[0:3]
ccinit = X[indinit,:]

kmbadfig = plt.figure(figsize=(10,3))

for i,its in enumerate([0,100]):
    if i==0:
        tstr = 'initial'
        km = cluster.KMeans(n_clusters=3, max_iter=1, n_init=1, init=ccinit)
        Yp = km.fit_predict(X)
        km.cluster_centers_ = ccinit # force clusters to initial points (hack)
    else:
        Yp = km.predict(X)
        km = cluster.KMeans(n_clusters=3, max_iter=its, n_init=1, init=ccinit)
        Yp = km.fit_predict(X)
```

```

tstr = 'iteration ' + str(its)

plt.subplot(1,2,i+1)
plot_clusters(km, axbox, X, Yp, rbow, rbow2)
plt.title(tstr)
plt.close()

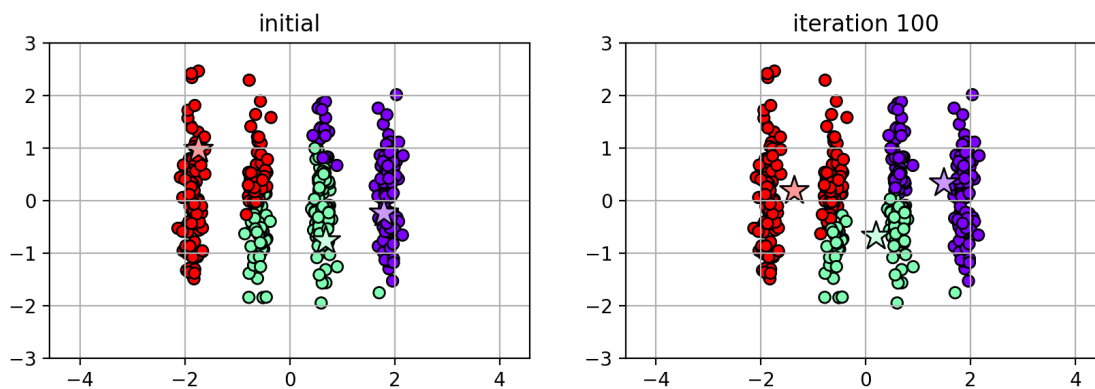
```

[35]: `kmbadfig`

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

[35]:



- **Solution:**

- Try several times using different initializations.
- Pick the answer with lowest objective score.

- In scikit-learn,

- 1) use a smart initialization method called “k-means++”, which speeds up convergence. (default)
- 2) use multiple random initializations.

```

[36]: # K-Means with 3 clusters (w/ Kmeans++ init)
km = cluster.KMeans(n_clusters=3, random_state=4487)
Yp = km.fit_predict(X)    # cluster data, and return labels

cc = km.cluster_centers_  # the cluster centers
cl = km.labels_           # labels also stored here

# K-means w/ random initialization (automatically does 10 trials)
kmr = cluster.KMeans(n_clusters=3, random_state=4487, init='random')
Ypr = kmr.fit_predict(X)  # cluster data, and return labels

```

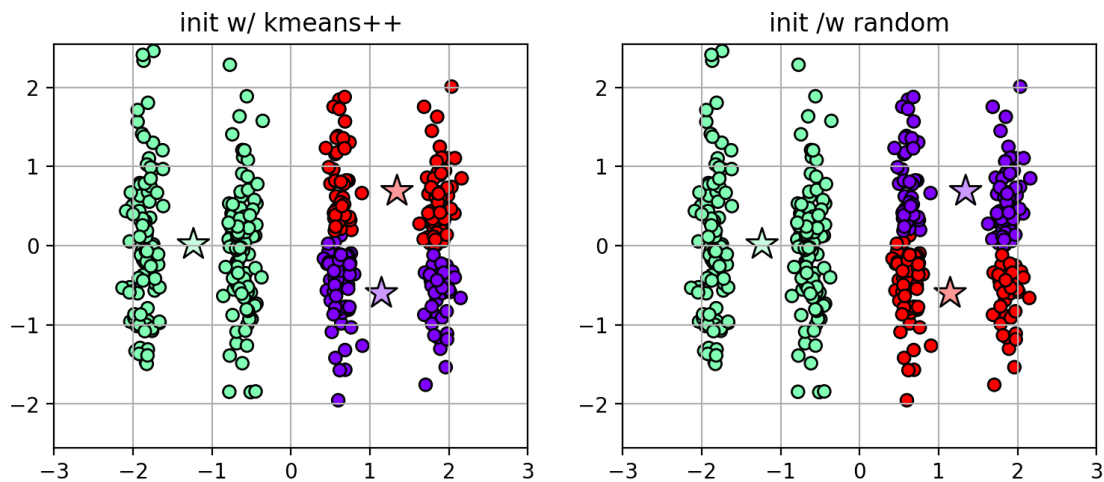
```
[37]: kmfig = plt.figure(figsize=(9,3.5))
plt.subplot(1,2,1)
plot_clusters(km, axbox, X, Yp, rbow, rbow2);
plt.title('init w/ kmeans++')
plt.subplot(1,2,2)
plot_clusters(kmr, axbox, X, Ypr, rbow, rbow2);
plt.title('init /w random');
plt.close()
```

```
[38]: kmfig
```

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

```
[38]:
```



14 For different K

- We need to choose the appropriate K

```
[39]: ksfig = plt.figure(figsize=(10,6))
for i,K in enumerate([2,3,4,5]):
    km = cluster.KMeans(n_clusters=K, random_state=4487)
    Yp = km.fit_predict(X)

    plt.subplot(2,2,i+1)
    plot_clusters(km, axbox, X, Yp, rbow, rbow2)
    plt.title(str(K) + ' clusters')
plt.close()
```



```
[40]: ksfig
```

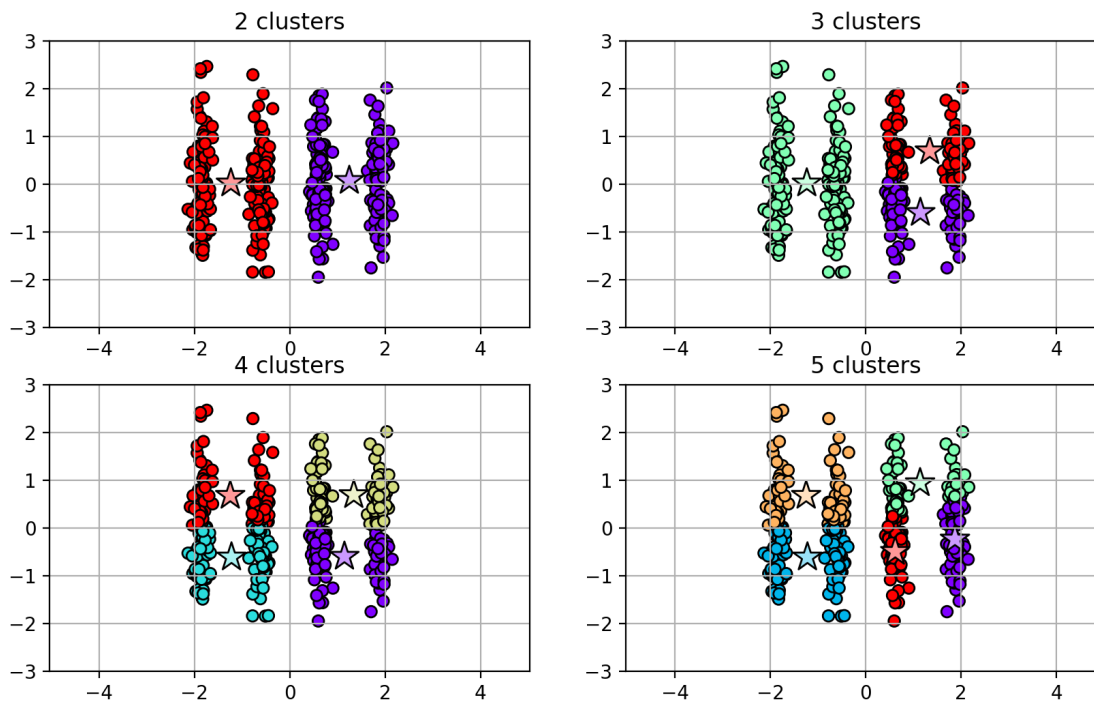
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

```
[40]:
```



- K-means partitions the input space into non-overlapping regions belonging to each cluster
 - assign each location in the space to the closest center.

```
[41]: krfig = plt.figure(figsize=(10,6))
for i,K in enumerate([2,3,4,5]):
    km = cluster.KMeans(n_clusters=K, random_state=4487)
    Yp = km.fit_predict(X)

    plt.subplot(2,2,i+1)
    plot_clusters(km, axbox, X, Yp, rbow, rbow2, showregions=True)
plt.close()
```

```
[42]: krfig
```

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data

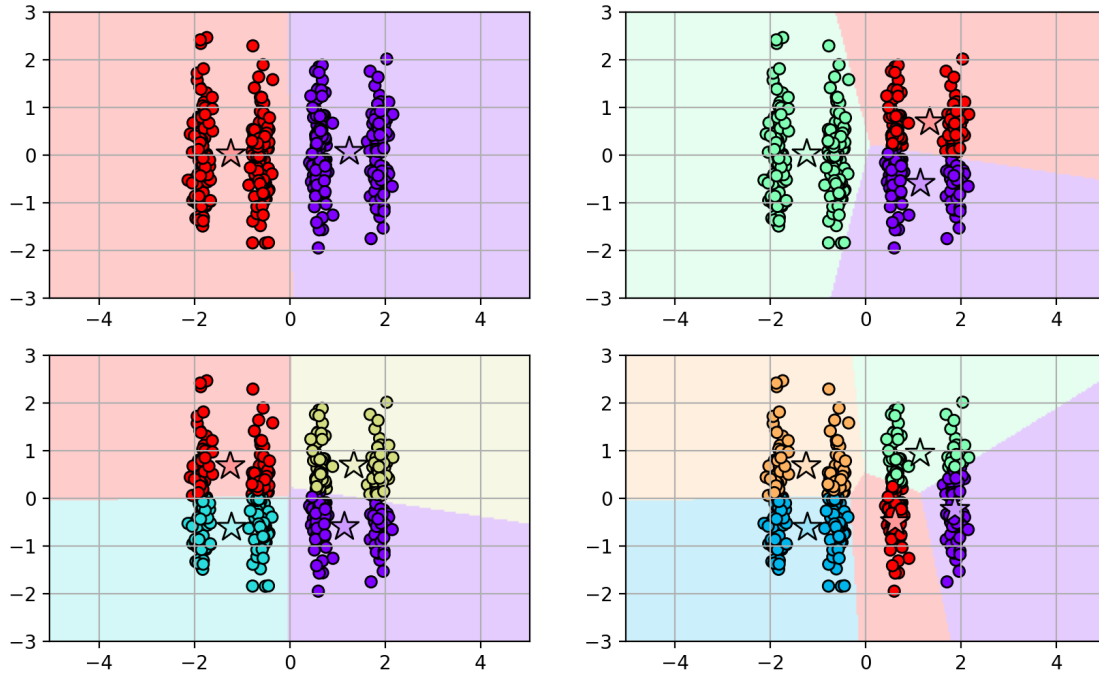
limits.

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

[42]:



15 Bag-of-X Representation

- K-means can partition the input space into regions.
- Create a new quantized representation for a set of samples.
 - 1) learn the partition space of the samples using K-means (i.e., the vocabulary)
 - 2) assign each sample to its closest center (i.e., the word)
 - 3) count the number of assignments for each center and form a histogram (i.e., the bag-of-words)
- Called a “bag-of-X” representation.
 - “X” is whatever modality you are using.

16 Example: Bag-of-visual-words

- Procedure:
 - 1) extract small patches from images
 - 2) learn the visual words using K-means
 - 3) assign patches to visual words, and form a histogram for each image.

- Use the bag-of-words model as the new feature vector.

```
[43]: # load images
oli = datasets.fetch_olivetti_faces(data_home="./")
img = oli.images
print(img.shape)

# extract 8x8 window patches with step size of 4x4
patches = skimage.util.view_as_windows(img, (1,8,8), step=(1,4,4))
print(patches.shape)

# reshape patches into 64-dim vectors
patchesall = patches.reshape((prod(patches.shape[0:3]),8*8))
print(patchesall.shape)

# run k-means (mini-batch version can handle large datasets)
# fit k-means, and predict the cluster index for each sample
K = 100
bows = cluster.MinibatchKMeans(n_clusters=K, random_state=5489, n_init=10)
wordsall = bows.fit_predict(patchesall)
```

```
(400, 64, 64)
(400, 15, 15, 1, 8, 8)
(90000, 64)
```

```
[44]: # function to make an image montage
def image_montage(X, imsize=None, maxw=10):
    """X can be a list of images, or a matrix of vectorized images.
    Specify imsize when X is a matrix."""
    tmp = []
    numimgs = len(X)

    # create a list of images (reshape if necessary)
    for i in range(0,numimgs):
        if imsize != None:
            tmp.append(X[i].reshape(imsize))
        else:
            tmp.append(X[i])

    # add blanks
    if (numimgs > maxw) and (mod(numimgs, maxw) > 0):
        leftover = maxw - mod(numimgs, maxw)
        meaning = 0.5*(X[0].max()+X[0].min())
        for i in range(0,leftover):
            tmp.append(ones(tmp[0].shape)*meaning)

    # make the montage
    tmp2 = []
```

```

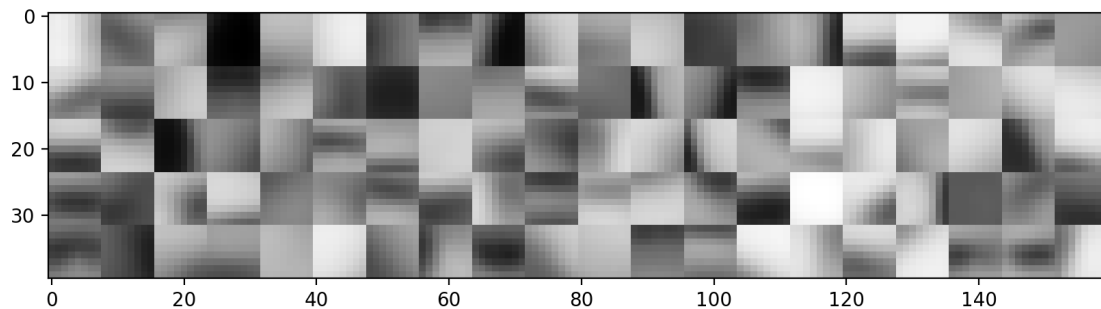
for i in range(0,len(tmp),maxw):
    tmp2.append( hstack(tmp[i:i+maxw]) )
montimg = vstack(tmp2)
return montimg

```

```

[45]: # get the visual words, and show them
visualwords = bows.cluster_centers_.reshape((K,8,8))
plt.figure(figsize=(10,4))
plt.imshow(image_montage(visualwords, maxw=20), cmap='gray',
↪interpolation='nearest');

```



```

[46]: # reshape the predicted words into small images
print(wordsall.shape)
patches_words = wordsall.reshape(patches.shape[0:3])
print(patches_words.shape)

# build the BoW histogram for each image
bowhist = [bincount(wds.ravel(), minlength=K) for wds in patches_words]

(90000,)
(400, 15, 15)

```

```

[47]: # visualize the BoW histogram and quantized image.
def show_bow_recon(visualwords, img, words, bowhist):
    plt.subplot(1,4,1)
    plt.imshow(img, cmap='gray')
    plt.title('image')

    plt.subplot(1,4,2)
    tmp = []
    for i in words:
        for j in i:
            tmp.append(visualwords[j])
    plt.imshow(image_montage(tmp, maxw=words.shape[1]), cmap='gray',
↪interpolation='nearest')

```

```

plt.title('visual words')

plt.subplot(1,4,3)
plt.imshow(words, cmap='jet')
plt.title('word indices')

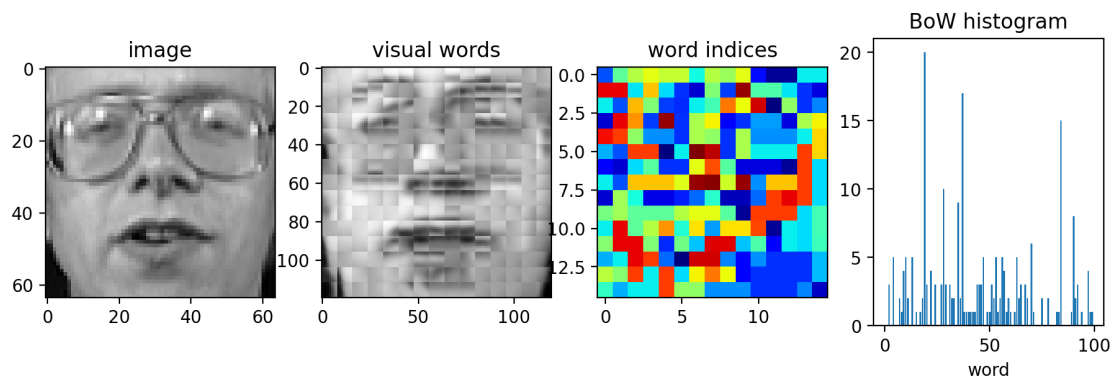
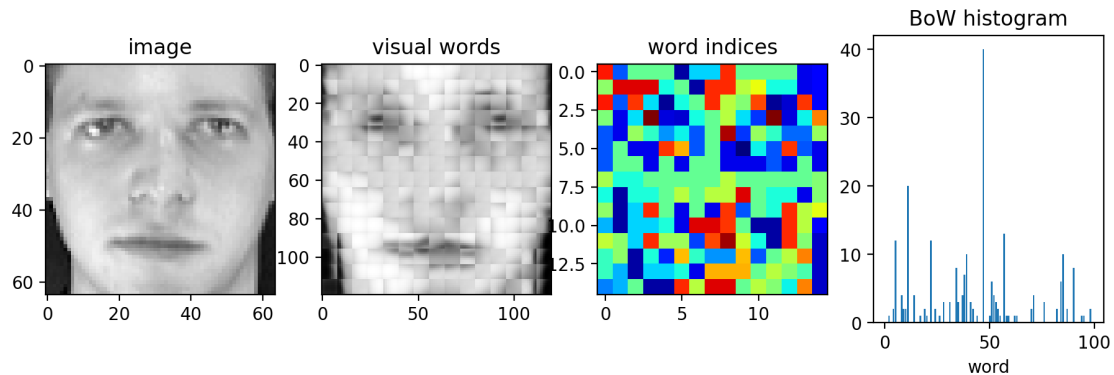
plt.subplot(1,4,4)
plt.bar(range(len(visualwords)), bowhist)
plt.title('BoW histogram')
plt.xlabel('word')

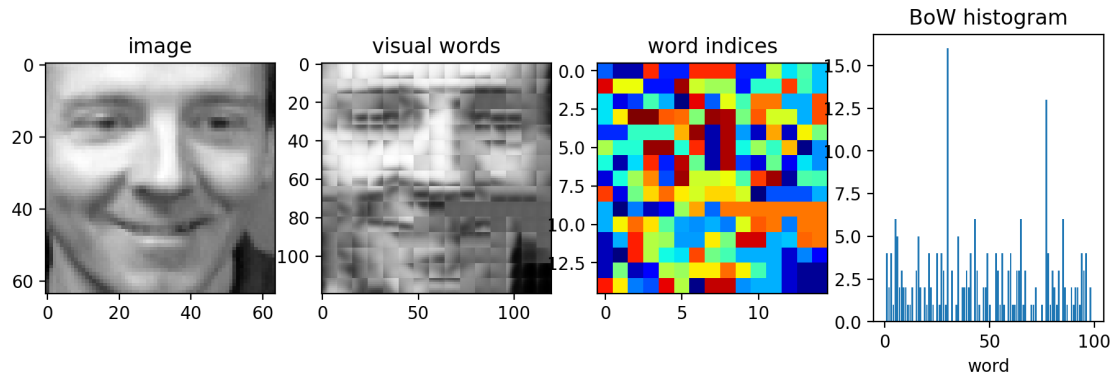
```

```

[48]: # view some reconstructions, words, and histograms
for t in [0,10,20]:
    plt.figure(figsize=(11,3))
    show_bow_recon(visualwords, img[t], patches_words[t], bowhist[t])
    plt.show()

```





17 Circular clusters

- One problem with K-means is that it assumes that each cluster has a circular shape.
 - based on Euclidean distance to each center
 - Kmeans cannot handle skewed (elliptical) clusters.

```
[49]: random.seed(38)
Xbars,Y = datasets.
      ↪make_blobs(400,2,centers=[[-15,0],[-5,0],[5,0],[15,0]],cluster_std=0.8)
Xbars[:,0] /=8

X = Xbars
axbox = [-3, 3, -3, 3]

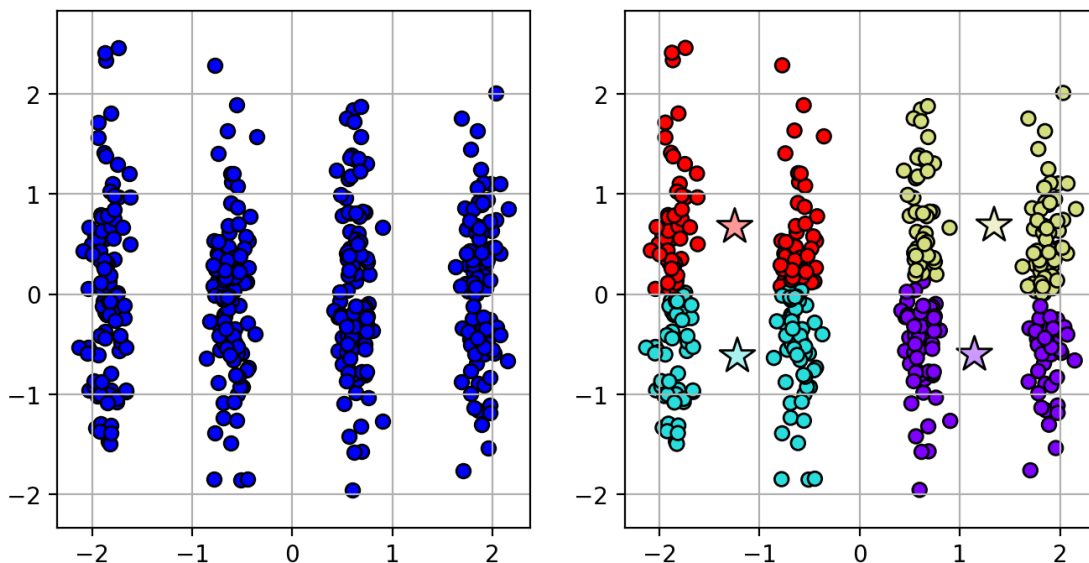
km = cluster.KMeans(n_clusters=4, random_state=4487)
Yp = km.fit_predict(X)

efig = plt.figure(figsize=(8,4))
plt.subplot(1,2,1)
plt.scatter(X[:,0], X[:,1], c='b', edgecolors='k')
plt.axis('equal'); plt.grid(True)

plt.subplot(1,2,2)
plot_clusters(km, axbox, X, Yp, rbow, rbow2)
plt.axis('equal'); plt.grid(True)
plt.close()
```

```
[50]: efig
```

```
[50]:
```



18 Gaussian mixture model (GMM)

- A multivariate Gaussian can model a cluster with an elliptical shape.
 - the ellipse shape is controlled by the covariance matrix of the Gaussian
 - the location of the cluster is controlled by the mean.
- Gaussian mixture model is a weighted sum of Gaussians

$$p(\mathbf{x}) = \sum_{j=1}^K \pi_j N(\mathbf{x} | \mu_j, \Sigma_j)$$

- Each Gaussian represents one elliptical cluster
 - * μ_j is the mean of the j-th Gaussian. (the location)
 - * Σ_j is the covariance matrix of the j-th Gaussian. (the ellipse shape)
 - * π_j is the prior weight of the j-th Gaussian. (how likely is this cluster)

19 1-D example of GMM

- $p(x) = 0.25 * N(x | -2, 4) + 0.75 * N(x | 3, 1)$
- each Gaussian is a “mountain”

```
[51]: eggmm = plt.figure(figsize=(6,3))

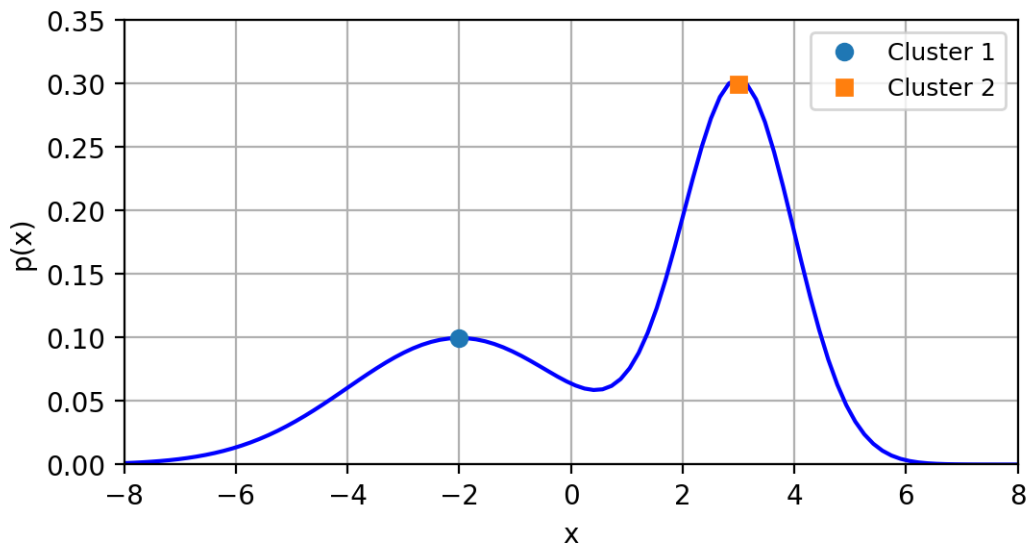
x = linspace(-8,8,100)
px = 0.25*(stats.norm.pdf((x+2)/2 )) + 0.75*(stats.norm.pdf(x-3))

plt.plot(x,px, 'b-')
plt.plot(-2,stats.norm.pdf(0)*0.25, 'o', label='Cluster 1')
```

```
plt.plot(3,stats.norm.pdf(0)*0.75, 's', label='Cluster 2')
plt.legend(loc=0, fontsize=9)
plt.xlabel('x'); plt.ylabel('p(x)')
plt.axis([-8, 8, 0, 0.35])
plt.grid(True)
plt.close()
```

[52]: eggmm

[52]:



20 2D example of GMM

- Each Gaussian defines a “mountain”
 - contours are ellipses

```
[53]: random.seed(38)
mu1 = array([0,0])
mu2 = array([6,8])
st1 = 2*random.normal(0,1,(2,2))
st2 = 2*random.normal(0,1,(2,2))
X1 = dot(random.normal(0,1,[60, 2]), st1.T) + mu1
X2 = dot(random.normal(0,1,[60, 2]), st2.T) + mu2
Xfull = r_[X1,X2]
axboxfull = [-10, 20, -7, 20]
gmmf = mixture.GaussianMixture(n_components=2, covariance_type='full',
                                random_state=4487, n_init=10)
gmmf.fit(Xfull)

efig = plt.figure(figsize=(10,6))
```



```

Yp = gmmf.predict(Xfull)
gmm2fig = plt.figure(figsize=(5,4.5))
plot_clusters(gmmf, axboxfull, None, None, rbow, rbow2)
plt.text(10,15,array2string(gmmf.means_[0], precision=2), color='purple')
plt.text(8,15,'$\mu_1$', color='purple')
plt.text(10,12,array2string(gmmf.covariances_[0], precision=2), color='purple')
plt.text(8,12.5,'$\Sigma_1$', color='purple')
plt.text(10,-2,array2string(gmmf.means_[1], precision=2), color='red')
plt.text(8,-2,'$\mu_2$', color='red')
plt.text(10,-5,array2string(gmmf.covariances_[1], precision=2), color='red')
plt.text(8,-4.5,'$\Sigma_2$', color='red')
plt.grid(True)
plt.close()

# save for later
X2class = Xfull.copy()

```

```

<>:19: SyntaxWarning: invalid escape sequence '\m'
<>:21: SyntaxWarning: invalid escape sequence '\S'
<>:23: SyntaxWarning: invalid escape sequence '\m'
<>:25: SyntaxWarning: invalid escape sequence '\S'
<>:19: SyntaxWarning: invalid escape sequence '\m'
<>:21: SyntaxWarning: invalid escape sequence '\S'
<>:23: SyntaxWarning: invalid escape sequence '\m'
<>:25: SyntaxWarning: invalid escape sequence '\S'
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_46775/4088641771.py:1
9: SyntaxWarning: invalid escape sequence '\m'
    plt.text(8,15,'$\mu_1$', color='purple')
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_46775/4088641771.py:2
1: SyntaxWarning: invalid escape sequence '\S'
    plt.text(8,12.5,'$\Sigma_1$', color='purple')
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_46775/4088641771.py:2
3: SyntaxWarning: invalid escape sequence '\m'
    plt.text(8,-2,'$\mu_2$', color='red')
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_46775/4088641771.py:2
5: SyntaxWarning: invalid escape sequence '\S'
    plt.text(8,-4.5,'$\Sigma_2$', color='red')

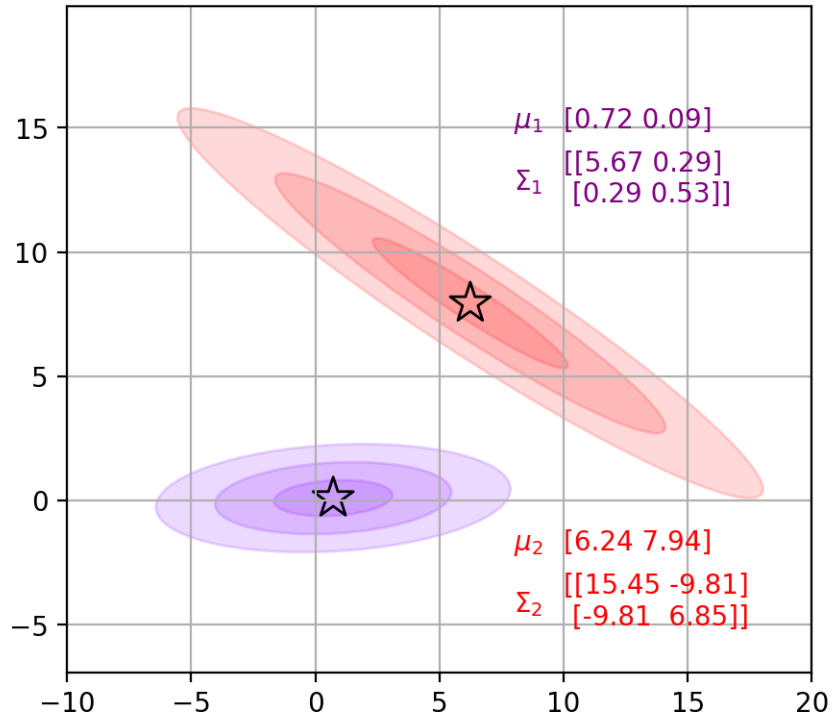
<Figure size 1000x600 with 0 Axes>

```

[54]: gmm2fig

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

[54]:



21 Clustering with GMMs

- Using the data, learn a GMM using maximum likelihood estimation:

$$\max_{\pi, \mu, \Sigma} \sum_{i=1}^N \log \sum_{j=1}^K \pi_j N(\mathbf{x}_i | \mu_j, \Sigma_j)$$

- The learned $\{\mu_j, \Sigma_j\}$ are the cluster center and ellipse shape.
- The learned π_j is the cluster population (percentage of points).
- This is a difficult to optimize because the “sum” is inside the “log”.

22 Expectation-Maximization (EM) algorithm for GMMs

- E-step:** Calculate cluster membership
 - assignment of point i to cluster j
 - * $\hat{z}_{ij} = p(z_i = j | \mathbf{x}_i) = \frac{\pi_j N(\mathbf{x}_i | \mu_j, \Sigma_j)}{\sum_k \pi_k N(\mathbf{x}_i | \mu_k, \Sigma_k)}$
 - uses “soft” assignment - a data point can have a fractional assignment to different clusters.
- M-step:** Update each Gaussian cluster (mean, covariance, and weight)
 - uses “soft” weighting
 - * “soft” count of points in cluster j : $N_j = \sum_{i=1}^N \hat{z}_{ij}$
 - * weight: $\pi_j = N_j / N$

- * mean: $\mu_j = \frac{1}{N_j} \sum_{i=1}^N \hat{z}_{ij} \mathbf{x}_i$
- * variance: $\Sigma_j = \frac{1}{N_j} \sum_{i=1}^N \hat{z}_{ij} (\mathbf{x}_i - \mu_j)^2$

- Similar to K-means, except uses “soft” assignments, rather than “hard” assignments.
- EM for GMM Example
 - red points - assigned to cluster 1
 - blue points - assigned to cluster 2
 - purple points - fractionally assigned between clusters 1 and 2

```
[55]: gmmits = []
      iters = [1,2,3, 5, 6, 10]
      for its in iters:
          gmmits.append(mixture.GaussianMixture(n_components=2,
                                                  covariance_type='full',
                                                  random_state=4487, n_init=1, max_iter=its,
                                                  init_params='random').fit(X2class))

      efig = plt.figure(figsize=(10,6))

      for i,gmm in enumerate(gmmits):
          Yp = gmm.predict(X2class)

          ax = plt.subplot(2,3,i+1)
          plot_clusters(gmm, axbox, X2class, Yp, mycmap, mycmap, showprob=True,
                        showcvlines=True)
          plt.axis('equal'); plt.grid(True)
          plt.title('iteration {}'.format(iters[i]))
          # grid of points
          #plt.subplot(2,3,i+1+3)
          #plot_clusters(gmm, axbox, X2class, Yp, rbow, rbow2, showregions=True)

      plt.tight_layout()
      plt.close()
```

```
/Users/zzs/miniconda3/envs/cs5489/lib/python3.12/site-
packages/sklearn/mixture/_base.py:275: ConvergenceWarning: Best performing
initialization did not converge. Try different init parameters, or increase
max_iter, tol, or check for degenerate data.
```

```
warnings.warn(
```

```
/Users/zzs/miniconda3/envs/cs5489/lib/python3.12/site-
packages/sklearn/mixture/_base.py:275: ConvergenceWarning: Best performing
initialization did not converge. Try different init parameters, or increase
max_iter, tol, or check for degenerate data.
```

```
warnings.warn(
```

```
/Users/zzs/miniconda3/envs/cs5489/lib/python3.12/site-
```

```
packages/sklearn/mixture/_base.py:275: ConvergenceWarning: Best performing
initialization did not converge. Try different init parameters, or increase
max_iter, tol, or check for degenerate data.
```

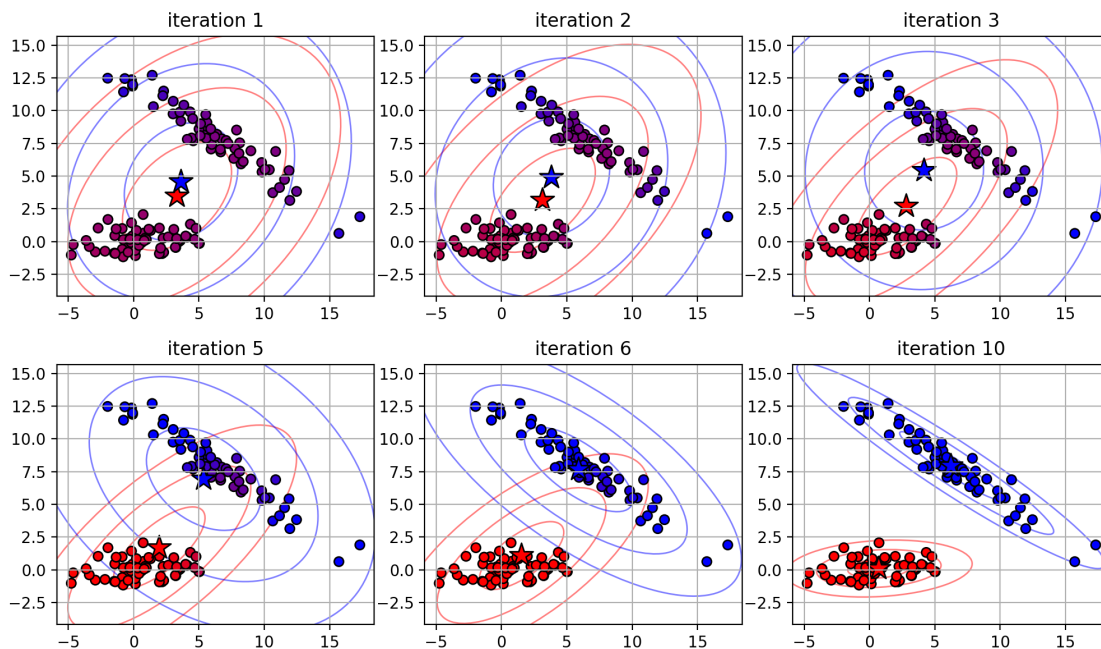
```
warnings.warn(
/Users/zzs/miniconda3/envs/cs5489/lib/python3.12/site-
packages/sklearn/mixture/_base.py:275: ConvergenceWarning: Best performing
initialization did not converge. Try different init parameters, or increase
max_iter, tol, or check for degenerate data.
```

```
warnings.warn(
/Users/zzs/miniconda3/envs/cs5489/lib/python3.12/site-
packages/sklearn/mixture/_base.py:275: ConvergenceWarning: Best performing
initialization did not converge. Try different init parameters, or increase
max_iter, tol, or check for degenerate data.
```

```
warnings.warn(
```

```
[56]: efig
```

```
[56]:
```



```
[57]: # fit a GMM
gmm = mixture.GaussianMixture(n_components=4, random_state=4487, n_init=10)

gmm.fit(X)
Y = gmm.predict(X)

cc = gmm.means_      # the cluster centers
```

```
[58]: efig = plt.figure(figsize=(12,4))
plt.subplot(1,3,1)
plt.scatter(X[:,0], X[:,1], c='b', edgecolors='k')
plt.grid(True)
plt.title('original data')

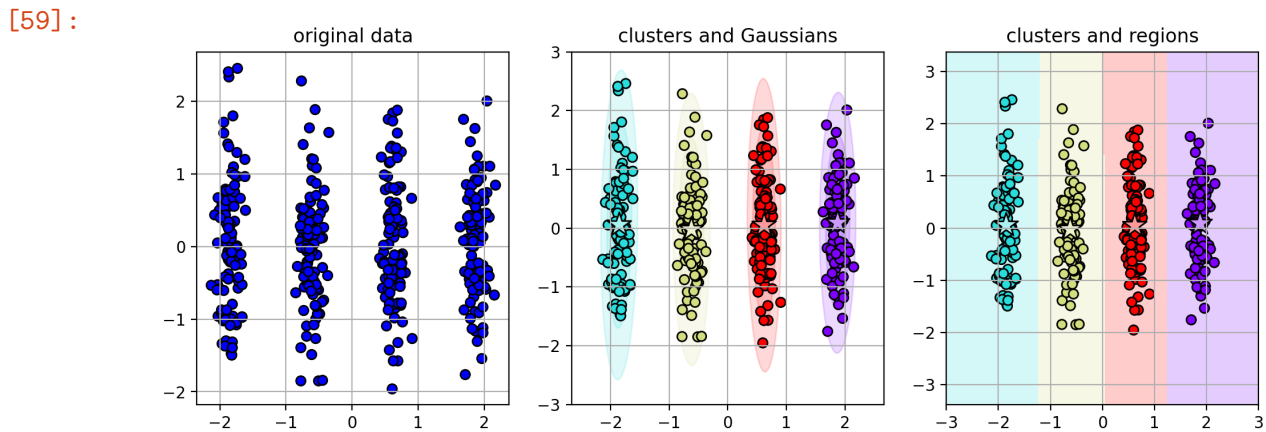
ax = plt.subplot(1,3,2)
plot_clusters(gmm, axbox, X, Y, rbow, rbow2)
plt.grid(True)
plt.title('clusters and Gaussians')

axbox = plt.axis()
plt.subplot(1,3,3)
plot_clusters(gmm, axbox, X, Y, rbow, rbow2, showregions=True)
plt.title('clusters and regions')
plt.close()
```

```
[59]: efig
```

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.



23 Covariance matrix

- The covariance matrix is a $d \times d$ matrix.

$$\Sigma_j = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

- For high-dimensional data, it can be very large.
 - requires a lot of data to learn effectively.

- Solution:
 - use *diagonal* covariance matrices (d parameters):

$$\Sigma_j = \text{diag}(\mathbf{a}) = \begin{bmatrix} a_1 & 0 & 0 \\ 0 & a_2 & 0 \\ 0 & 0 & a_3 \end{bmatrix}$$

- * Axes of ellipses will be aligned with the axes.
 - use *spherical* covariance matrices (1 parameter)

$$\Sigma_j = a\mathbf{I} = \begin{bmatrix} a & 0 & 0 \\ 0 & a & 0 \\ 0 & 0 & a \end{bmatrix}$$

- * Clusters will be circular (similar to K-means)

```
[60]: random.seed(38)
mu1 = array([0,0])
mu2 = array([6,8])
st1 = 2*random.normal(0,1,(2,2))
st2 = 2*random.normal(0,1,(2,2))
X1 = dot(random.normal(0,1,[60, 2]), st1.T) + mu1
X2 = dot(random.normal(0,1,[60, 2]), st2.T) + mu2
Xfull = r_[X1,X2]

X = Xfull
axbox = [-10, 20, -7, 20]
```

```
[61]: # full covariance (d*d parameters)
gmmf = mixture.GaussianMixture(n_components=2,
                                covariance_type='full',
                                random_state=4487, n_init=10)

gmmf.fit(X)

# diagonal covariance (d parameters)
gmmd = mixture.GaussianMixture(n_components=2,
                                covariance_type='diag',
                                random_state=4487, n_init=10)

gmmd.fit(X)

# spherical covariance (1 parameter)
gmms = mixture.GaussianMixture(n_components=2,
                                covariance_type='spherical',
                                random_state=4487, n_init=10)

gmms.fit(X)
```

```
[61]: GaussianMixture(covariance_type='spherical', n_components=2, n_init=10,
                        random_state=4487)
```

```
[62]: efig = plt.figure(figsize=(10,6))

for i,gmm in enumerate([gmmf, gmmd, gmms]):
    Yp = gmm.predict(X)

    ax = plt.subplot(2,3,i+1)
    plot_clusters(gmm, axbox, X, Yp, rbow, rbow2)
    plt.axis('equal'); plt.grid(True)
    plt.title(gmm.covariance_type)

    # grid of points
    plt.subplot(2,3,i+1+3)
    plot_clusters(gmm, axbox, X, Yp, rbow, rbow2, showregions=True)

plt.close()
```

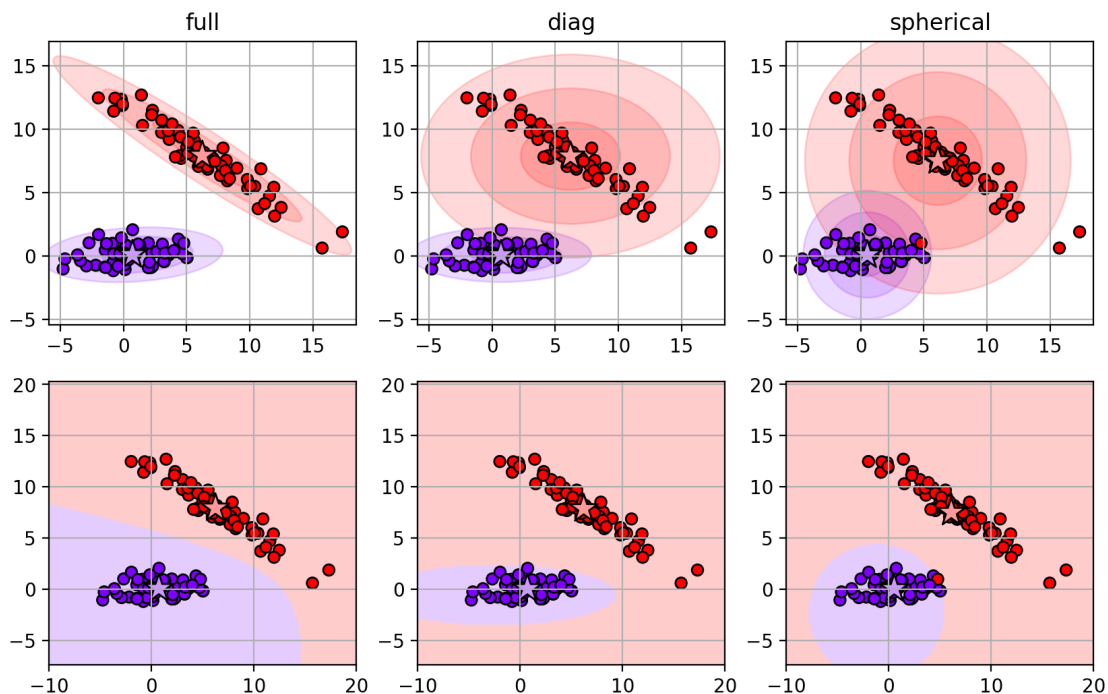
```
[63]: efig
```

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

```
[63]:
```



24 Expectation Maximization (EM) Algorithm

- An general algorithm for finding the MLE solution when there are hidden (unseen) variables z .
 - Goal: $\hat{\theta} = \operatorname{argmax}_{\theta} \log p_{\theta}(\mathbf{x})$
 - * where $p_{\theta}(\mathbf{x}) = \sum_z p(\mathbf{x}|z)p(z)$
 - * θ are the parameters.
- Solution:
 - iterate between estimating the hidden variables z and maximizing w.r.t the parameters θ .
- For GMMs:
 - z is the assignment of $\mathbf{x}$ to one of the Gaussian components.
 - prior probability: $p(z = j) = \pi_j$
 - Gaussian component: $p(\mathbf{x}|z = j) = N(\mathbf{x}|\mu_j, \Sigma_j)$
 - likelihood: $p(\mathbf{x}) = \sum_z p(\mathbf{x}|z)p(z) = \sum_j \pi_j N(\mathbf{x}|\mu_j, \Sigma_j)$

25 EM Algorithm

- 1) Select an initial model $\hat{\theta}$
- 2) **E-step**: estimate the hidden variables as their expected value.
 - $Q(\theta) = E_{z|\mathbf{x}, \hat{\theta}}[\log p_{\theta}(\mathbf{x}, z)]$
 - the expectation uses current model parameters $\hat{\theta}$.
- 3) **M-step**: maximize w.r.t θ
 - $\hat{\theta} = \operatorname{argmax}_{\theta} Q(\theta)$
- 4) repeat E- and M-steps until convergence.
 - It's gauranteed to converge.

26 Joint likelihood for GMMs

- Joint likelihood of $(\mathbf{x}_i, z_i)$

$$p(\mathbf{x}_i, z_i) = p(\mathbf{x}_i|z_i)p(z_i) = \pi_{z_i} N(\mathbf{x}_i|\mu_{z_i}, \Sigma_{z_i})$$

- annoying because z_i is indexing the parameters.

27 Indicator Variable Trick

- Define z_{ij} as the indicator variable that $z_i = j$
 - $z_{ij} = \begin{cases} 1, & z_i = j \\ 0, & \text{otherwise} \end{cases}$
- Joint likelihood of $(\mathbf{x}_i, z_i)$

- z_{ij} selects the correct Gaussian component.

$$p(\mathbf{x}_i, z_i) = \prod_{j=1}^K (p(\mathbf{x}_i | z_i = j) p(z_i = j))^{z_{ij}} = \prod_{j=1}^K (\pi_j N(\mathbf{x}_i | \mu_j, \Sigma_j))^{z_{ij}}$$

$$\Rightarrow \log p(\mathbf{x}_i, z_i) = \sum_{j=1}^K z_{ij} \log(\pi_j N(\mathbf{x}_i | \mu_j, \Sigma_j))$$

28 EM algorithm for GMMs revisited

- **E-step:** calculate the Q function as expectation of hidden variables:

$$\begin{aligned} Q(\theta) &= E_{z|\mathbf{x}, \hat{\theta}}[\log p_{\theta}(\mathbf{x}, z)] \\ &= E_{z|\mathbf{x}, \hat{\theta}} \left[\sum_{j=1}^K z_{ij} \log(\pi_j N(\mathbf{x}_i | \mu_j, \Sigma_j)) \right] \\ &= \sum_{j=1}^K E_{z|\mathbf{x}, \hat{\theta}}[z_{ij}] \log(\pi_j N(\mathbf{x}_i | \mu_j, \Sigma_j)) \end{aligned}$$

- Define the expectation of z_{ij} :
 - $\hat{z}_{ij} = E_{z|\mathbf{x}, \hat{\theta}}[z_{ij}] = p(z_i = j | \mathbf{x}_i, \hat{\theta}) = \frac{\hat{\pi}_j N(\mathbf{x}_i | \hat{\mu}_j, \hat{\Sigma}_j)}{\sum_k \hat{\pi}_k N(\mathbf{x}_i | \hat{\mu}_k, \hat{\Sigma}_k)}$
- “soft” assignment based on the current GMM estimate $\hat{\theta} = \{\hat{\pi}_j, \hat{\mu}_j, \hat{\Sigma}_j\}_j$
- **M-step:** maximize the Q function w.r.t. θ .

$$\begin{aligned} \hat{\theta} &= \underset{\theta}{\operatorname{argmax}} Q(\theta) \\ &= \underset{\theta}{\operatorname{argmax}} \sum_{j=1}^K \hat{z}_{ij} \log(\pi_j N(\mathbf{x}_i | \mu_j, \Sigma_j)) \end{aligned}$$

- maximize w.r.t. each parameter:
 - weight: $\pi_j = N_j / N$
 - mean: $\mu_j = \frac{1}{N_j} \sum_{i=1}^N \hat{z}_{ij} \mathbf{x}_i$
 - variance: $\Sigma_j = \frac{1}{N_j} \sum_{i=1}^N \hat{z}_{ij} (\mathbf{x}_i - \mu_j)^2$
 - “soft” count of points in cluster j : $N_j = \sum_{i=1}^N \hat{z}_{ij}$

29 How to select K ?

- Clustering results depends on the number of clusters used.
- We don’t typically know this information beforehand.

30 Bayesian GMM

- A Bayesian version of GMM that automatically selects the value of K
 - use a Dirichlet distribution to model the prior probabilities, $p(\pi)$.
- *concentration* parameter α controls the range of π values that are preferred

- higher values encourage more clusters (π_j are uniform)
- lower values encourage less clusters (more $\pi_j = 0$)
- Mathematically, α adds to the number of samples in the cluster, $\pi_j = \frac{N_j + \alpha}{N + K\alpha}$
 - * similar to Laplace smoothing.
- (more details in the lecture on graphical models)

```
[64]: # alpha = concentration parameter
# n_components = the max number of components to consider
dpgmm = mixture.BayesianGaussianMixture(covariance_type='full',
                                         weight_concentration_prior=1,
                                         n_components=5, max_iter=100, random_state=4487)

dpgmm.fit(X)
Y = dpgmm.predict(X)
cl = unique(Y)          # find active clusters
newK = len(cl)          # number of clusters
cc = dpgmm.means_[cl]   # get means
```

```
[65]: dfig = plt.figure(figsize=(10,4))
plt.subplot(1,3,1)
plt.scatter(X[:,0],X[:,1], c='b', edgecolors='k')
plt.axis('equal'); plt.grid(True)

ax = plt.subplot(1,3,2)
Knew = plot_clusters(dpgmm, axbox, X, Y, rbow, rbow2)
plt.axis('equal'); plt.grid(True)
plt.title('BayesGMM (' + str(dpgmm.n_components) + " -> " + str(Knew[0]) + "
↳clusters)")

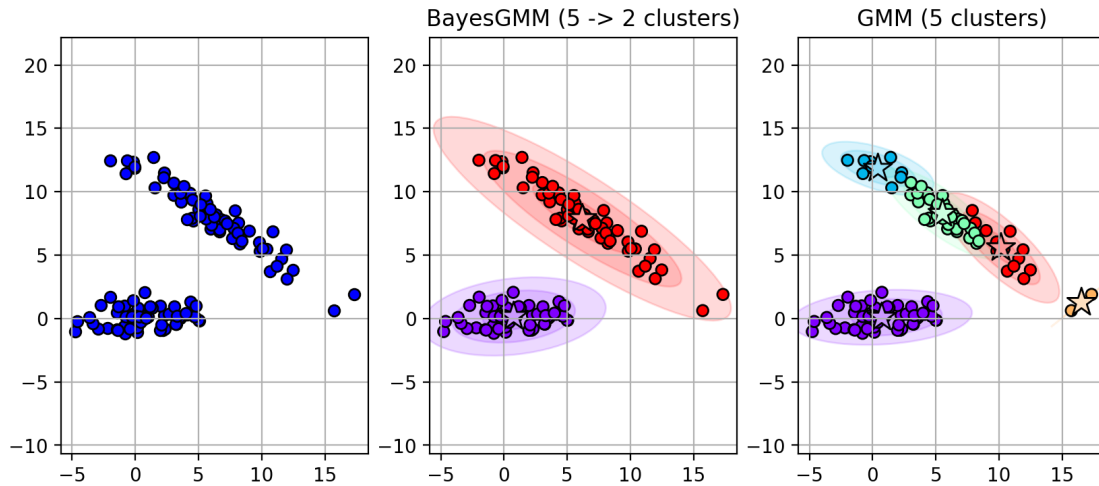
gmm = mixture.GaussianMixture(n_components=5, covariance_type='full',
↳random_state=4487, n_init=10)
gmm.fit(X)
Y2 = gmm.predict(X)

plt.subplot(1,3,3)
plot_clusters(gmm, axbox, X, Y2, rbow, rbow2)
plt.axis('equal'); plt.grid(True)
plt.title('GMM (' + str(gmm.n_components) + " clusters)")
plt.close()
```

- Bayesian GMM automatically selects 2 components from 5
 - for comparison, GMM with 5 clusters looks messy

```
[66]: dfig
```

```
[66]:
```



```
[67]: # make some random data
Xblob = array([])
random.seed(120)
for i in range(5):
    mu = random.uniform(-5,5,(1,2))

    st = 0.2*random.normal(0,1,(2,2)) + 0.2*identity(2)
    X1 = dot(random.normal(0,1,[60, 2]), st.T) + mu

    if i==0:
        Xblob = X1
    else:
        Xblob = r_[Xblob,X1]
X = Xblob
axbox = [-6, 6, -7, 2]
```

- For different concentration parameter α
 - larger α may yield more clusters

```
[68]: dpfig = plt.figure(figsize=(10,6))
plt.subplot(2,3,1)
plt.scatter(X[:,0], X[:,1], c='b', edgecolors='k')
plt.axis(axbox); plt.grid(True)

for i,alpha in enumerate(logspace(-5,3,5)):
    dpgmm = mixture.BayesianGaussianMixture(covariance_type='full',
                                             weight_concentration_prior=alpha,
                                             n_components=10, max_iter=100, random_state=6487)
    dpgmm.fit(X)
    Y = dpdgm.predict(X)
```

```

plt.subplot(2,3,i+2)
Knew = plot_clusters(dpgmm, axbox, X, Y, rbow, rbow2)
plt.grid(True)
plt.title("$\\alpha$="+str(alpha)+"; " + str(Knew[0])+ " clusters")
plt.tight_layout()
plt.close()

```

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

[69]: dpfig

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

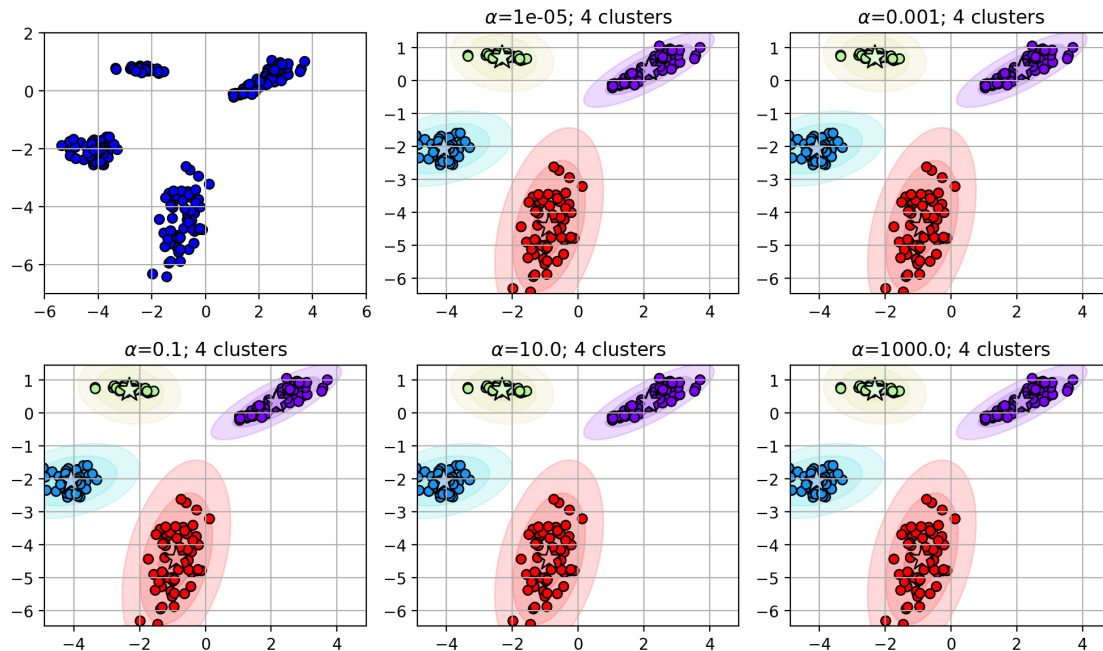
Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

[69]:



- Choice of α is not that critical
 - same number of clusters for large ranges of α

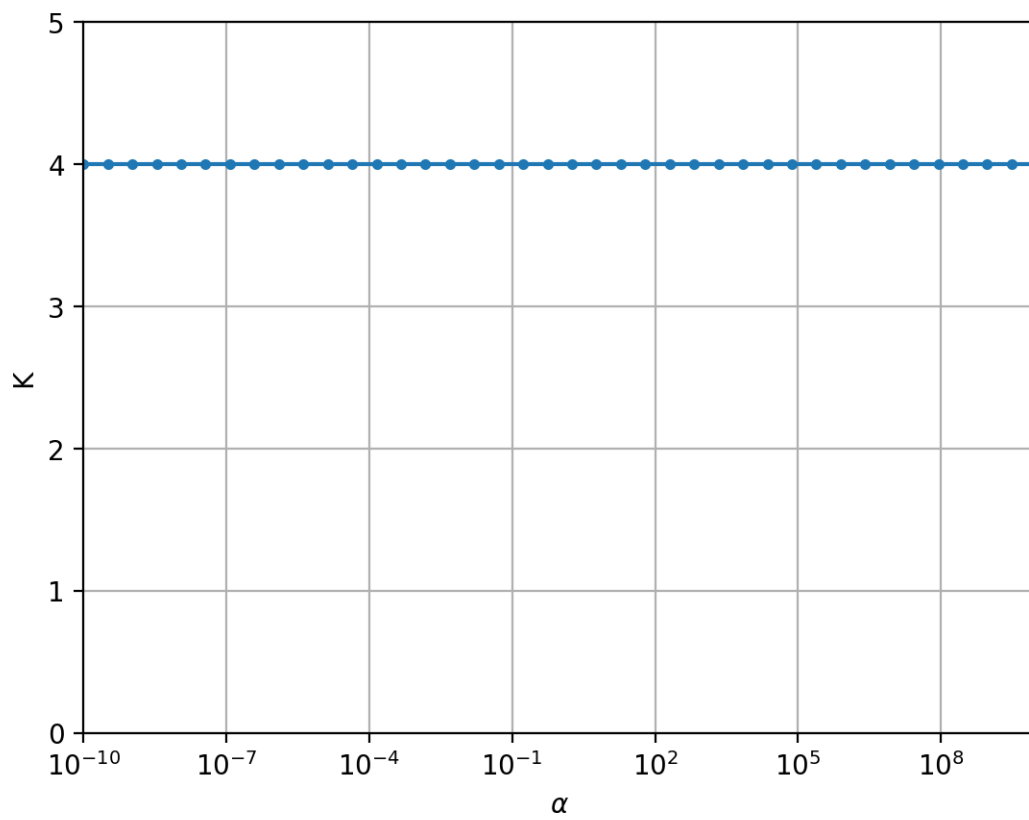
```
[70]: alphas = logspace(-10,10,40)
allKs = zeros(alphas.shape)
for i,alpha in enumerate(alphas):
    dpgmm = mixture.BayesianGaussianMixture(covariance_type='full',
    ↪weight_concentration_prior=alpha,
                                n_components=10, max_iter=100,
                                random_state=6487)

    dpgmm.fit(X)
    allKs[i] = len(unique(dpgmm.predict(X)))

dpgmmafig = plt.figure()
plt.semilogx(alphas, allKs, '.-', drawstyle='steps-post')
plt.grid(True)
plt.axis([min(alphas), max(alphas), 0, 5]);
plt.xlabel('$\alpha$'); plt.ylabel('K')
plt.close()
```

```
[71]: dpgmmafig
```

```
[71]:
```



[]:

Lecture9b

October 31, 2025

1 CS5489 - Machine Learning

2 Lecture 9b - Unsupervised Learning - Clustering

2.0.1 Dept. of Computer Science, City University of Hong Kong

3 Outline

1. Unsupervised Learning
2. Parametric clustering
 1. K-means
 2. Gaussian mixture models (GMMs)
 3. Bayesian GMMs
3. **Non-parametric clustering and Mean-shift**
4. Spectral clustering

```
[1]: # setup
%matplotlib inline
import matplotlib_inline # setup output image format
matplotlib_inline.backend_inline.set_matplotlib_formats('retina')
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 100 # display larger images
import matplotlib
from numpy import *
from sklearn import *
from scipy import stats
from mpl_toolkits.mplot3d import Axes3D
```

```
[2]: random.seed(38)
mu1 = array([0,0])
mu2 = array([6,8])
st1 = 2*random.normal(0,1,(2,2))
st2 = 2*random.normal(0,1,(2,2))
X1 = dot(random.normal(0,1,[60, 2]), st1.T) + mu1
X2 = dot(random.normal(0,1,[60, 2]), st2.T) + mu2
Xfull = r_[X1,X2]
axboxfull = [-10, 20, -7, 20]
```

```

X = Xfull
axbox = axboxfull
ssize = 16

```

```

[3]: mycmap = matplotlib.colors.ListedColormap(["#FF0000", "#00FF00", "#0000FF"])
cols = ['r', 'g', 'b']
jet = plt.get_cmap('jet')
rbow = plt.get_cmap('rainbow')

al = 0.4
rbow_data2 = {
    'red': lambda x: (1-al) + al*abs(2 * x - 0.5),
    'green': lambda x: (1-al) + al*sin(x * pi),
    'blue': lambda x: (1-al) + al*cos(x * pi / 2),
}
rbow2 = matplotlib.colors.LinearSegmentedColormap('rainbow2', rbow_data2)

# ax is the axis
# cmap is colormap to use
# cl is the index of components to show [default is None which means all
↪components]
def make_ellipses(means, covars, ax, cmap, cl=None, alphamul=1.,
    ↪showedge=False):
    dim = means.shape[1]
    if cl is None:
        cl = range(means.shape[0])
    for n in cl:
        mycv = covars[n]
        if mycv.size==1:
            mycv = mycv*eye(dim)
        elif mycv.size==dim:
            mycv = diag(mycv)

        v, w = linalg.eigh(mycv)

        u = w[0] / linalg.norm(w[0])
        angle = arctan2(u[1], u[0])
        angle = 180 * angle / pi # convert to degrees
        s = sqrt(v) # stddev

        for f in [1,2,3]:
            ss = s*f
            eopts = {}
            if showedge:
                eopts['edgecolor'] = 'k'
            else:
                eopts['edgecolor'] = None

```



```

        ell = matplotlib.patches.Ellipse(means[n, :2], 2*ss[0], 2*ss[1],
                                           angle=180 + angle,
                                           facecolor=cmap(float(n)/(lambda a,b: a if a>b else b)(max(c1),1)),
                                           **eopts)
        ell.set_clip_box(ax.bbox)
        ell.set_alpha(0.15*alphanul)
        ax.add_artist(ell)

def plot_clusters(model, axbox, X, Y, pcmap, ccmap, showregions=False):
    # model = kmeans, gmm, dpvgmm
    # X = points
    # pcmap = colormap for points
    # ccmap = colormap for centers

    name = model.__class__.__name__

    if (name == 'KMeans'):
        cc = model.cluster_centers_
        cl = range(model.n_clusters)
        cv = None
    elif (name == 'GMM') or (name == 'GaussianMixture'):
        cc = model.means_
        cl = range(model.n_components)
        cv = model.covariances_
    elif (name == 'DPGMM') or (name == 'BayesianGaussianMixture'):
        cl = unique(Y) # find active clusters
        cc = model.means_[cl] # get means
        cv = model.covariances_[cl]
    elif (name == 'VBGMM'):
        cl = unique(Y) # find active clusters
        cc = model.means_[cl] # get means
        cv = None
    elif (name == 'MeanShift'):
        cc = model.cluster_centers_
        cl = range(len(cc))
        cv = None
    elif (name == 'SpectralClustering'):
        cc = None
        cl = range(model.n_clusters)
        cv = None
    elif (name == 'DBSCAN'):
        cc = None
        cv = None
        cl = unique(model.labels_)
        cl = cl[cl != -1]
        core_mask = zeros_like(Y, dtype=bool)
        core_mask[model.core_sample_indices_] = True

```

```

outlier_mask = (Y == -1)
noncore_mask = logical_not(core_mask) & logical_not(outlier_mask)

if showregions:
    dx = axbox[1]-axbox[0]
    dy = axbox[3]-axbox[2]

    axbox2 = [ axbox[0]-0.5*dx, axbox[1]+0.5*dx,
               axbox[2]-0.5*dy, axbox[3]+0.5*dy]

    # grid of points
    xr = [linspace(axbox2[0],axbox2[1],300),
          linspace(axbox2[2],axbox2[3],300)]
    xgrid0, xgrid1 = meshgrid(xr[0], xr[1])
    allpts = c_[xgrid0.ravel(), xgrid1.ravel()]

    # predictions
    Ypred = model.predict(allpts).reshape(xgrid0.shape)

    plt.imshow(Ypred, origin='lower', extent=axbox2, aspect='auto',
               vmin=min(c1), vmax=max(c1), alpha=0.2, cmap=pcmap)

    # show ellipses
    if (cv is not None) and (not showregions):
        make_ellipses(cc, cv, plt.gca(), pcmap, None)

    # show points
    if X is not None:
        if (name == 'DBSCAN'):
            plt.scatter(X[core_mask,0], X[core_mask,1], c=Y[core_mask],
                cmap=pcmap, label='core points', edgecolors='k')
            plt.scatter(X[noncore_mask,0], X[noncore_mask,1], c=Y[noncore_mask],
                cmap=pcmap, label='border points',
                vmin=min(c1), vmax=max(c1), marker='v', edgecolors='k')
            plt.scatter(X[outlier_mask,0], X[outlier_mask,1], color='k',
                label='noise', s=6, edgecolors='k')
        else:
            plt.scatter(X[:,0], X[:,1], c=Y, cmap=pcmap, edgecolors='k',
                s=ssize)

    # show centers
    if cc is not None:
        plt.scatter(cc[:,0], cc[:,1], c=c1, s=250, cmap=ccmap, marker='*',
            edgecolors='k')

plt.axis('equal'); plt.axis(axbox); plt.grid(True)

```

```
# return number of clusters found
return (len(cl),)
```

```
[4]: def plot_scores(model, axbox, mycmap, showlabels=True, showcontour=True):
    xr = [ linspace(axbox[0], axbox[1], 200),
           linspace(axbox[2], axbox[3], 200) ]

    # make a grid for calculating the posterior,
    # then form into a big [N,2] matrix
    xgrid0, xgrid1 = meshgrid(xr[0], xr[1])
    allpts = c_[xgrid0.ravel(), xgrid1.ravel()]

    # calculate scores
    prob = model.score_samples(allpts)
    prob1 = exp(prob.reshape(xgrid0.shape))

    # contour plot of the posterior and decision boundary
    plt.imshow(prob1, origin='lower', extent=axbox, alpha=0.50, cmap=mycmap)
    if showlabels:
        plt.colorbar(shrink=0.6)
    #levels=[0.1, 0.3, 0.7, 0.9]
    if showcontour:
        CS = plt.contour(xr[0], xr[1], prob1, cmap=mycmap, alpha=0.8)
    # if showlabels:
    #     #plt.colorbar(CS)
    #     plt.clabel(CS, inline=1, fontsize=10)
    #     plt.contour(xr[0], xr[1], post1, levels=[0.5], linestyle='dashed',
    ↪ colors='black')
```

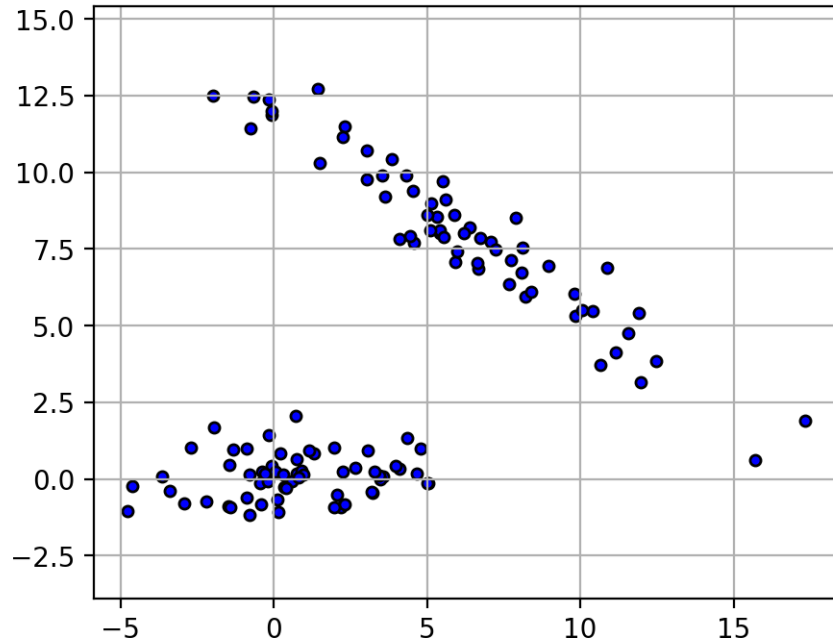
4 Non-parametric densities

- Suppose we have samples $\{\mathbf{x}_1, \dots, \mathbf{x}_N\}$
 - We want to estimate a probability density without assuming a parametric model (e.g., Gaussian)

```
[5]: X = Xfull
fig = plt.figure(figsize=(5,4))
plt.scatter(X[:,0], X[:,1], c='b', s=ssize, edgecolors='k')
plt.axis(axbox)
plt.axis('equal')
plt.grid(True)
plt.close()
```

```
[6]: fig
```

```
[6]:
```



5 Non-parametric estimation

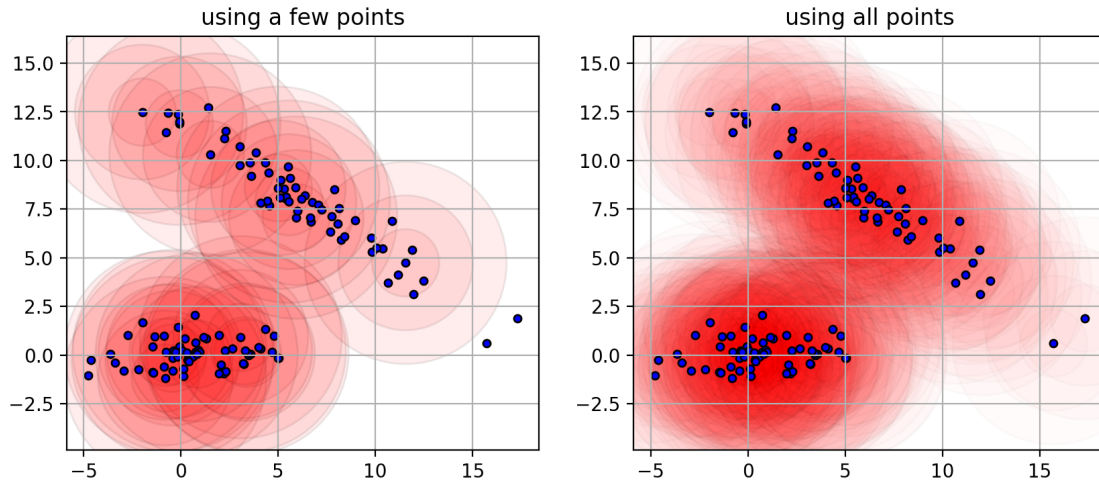
- Idea: put a small Gaussian at each data point, and sum it up.
 - each point contributes locally to the probability density.
 - $p(\mathbf{x}) = \frac{1}{N} \sum_{i=1}^N N(\mathbf{x}|\mathbf{x}_i, \sigma^2 \mathbf{I})$
 - * σ is the bandwidth of the Gaussian.

```
[7]: fig = plt.figure(figsize=(10,4))
mycmap = matplotlib.colors.ListedColormap(["#FF0000"])
plt.subplot(1,2,1)
make_ellipses(X[::8], full((len(X[::8])),), 3.), plt.gca(), mycmap, alphamul=0.
↪5, showedge=True)
plt.scatter(X[:,0], X[:,1], c='b', s=ssize, edgecolors='k')
plt.axis(axbox)
plt.axis('equal')
plt.grid(True)
plt.title('using a few points')
plt.subplot(1,2,2)
make_ellipses(X, full((len(X)),), 3.), plt.gca(), mycmap, alphamul=0.1,
↪showedge=True)
plt.scatter(X[:,0], X[:,1], c='b', s=ssize, edgecolors='k')
plt.axis(axbox)
plt.axis('equal')
plt.grid(True)
```

```
plt.title("using all points")
plt.close()
```

```
[8]: fig
```

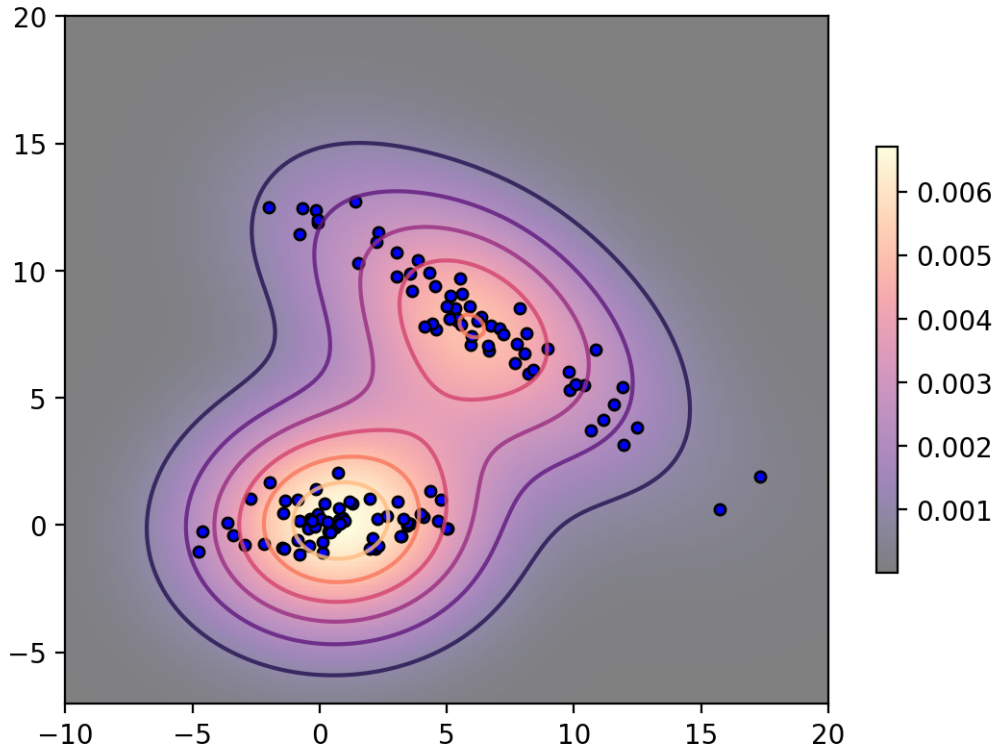
```
[8]:
```



6 Kernel density estimator

- This is called a *kernel density estimator*
 - the *kernel* is the small Gaussian.

```
[9]: kde = neighbors.KernelDensity(bandwidth=3.0).fit(X)
plot_scores(kde, axbox, 'magma')
plt.scatter(X[:,0], X[:,1], c='b', s=ssize, edgecolors='k');
```



7 Clustering using KDE

- The modes of the KDE can be considered the cluster centers.
 - A mode is a local maximum of the probability density.
 - The number of clusters is selected automatically according to the data.
- *How to find the cluster centers?*
 - mode: $\mu = \operatorname{argmax}_x p(\mathbf{x})$
 - select a point, and run gradient ascent on $p(\mathbf{x})$.
 - * $\hat{\mu} \leftarrow \hat{\mu} + \eta \frac{d}{d\mathbf{x}} p(\mathbf{x})$
 - using a clever choice of η , we get an algorithm that is guaranteed to converge called the “Mean-shift algorithm”.

8 Mean-shift algorithm

- **Idea:** iteratively shift towards the largest concentration of points.
 - start from an initial point $\mathbf{x}$ (e.g., one of the data points).
 - repeat until $\mathbf{x}$ is unchanged:
 - 1) find the nearest neighbors to $\mathbf{x}$ within some radius (bandwidth)
 - * according to the Gaussian kernels.
 - 2) set $\mathbf{x}$ to be the mean of the neighbor points.

```
[10]: X = Xfull
x0 = X[3,:]-array([-2,0])

efigs = []
for i in range(8):
    if i % 2 == 0:
        efigs.append(plt.figure(figsize=(10,4)))

    # need to use the function version which can set the number of iterations
    cc,cl = cluster.mean_shift(X, bandwidth=3, max_iter=0, seeds=[x0])

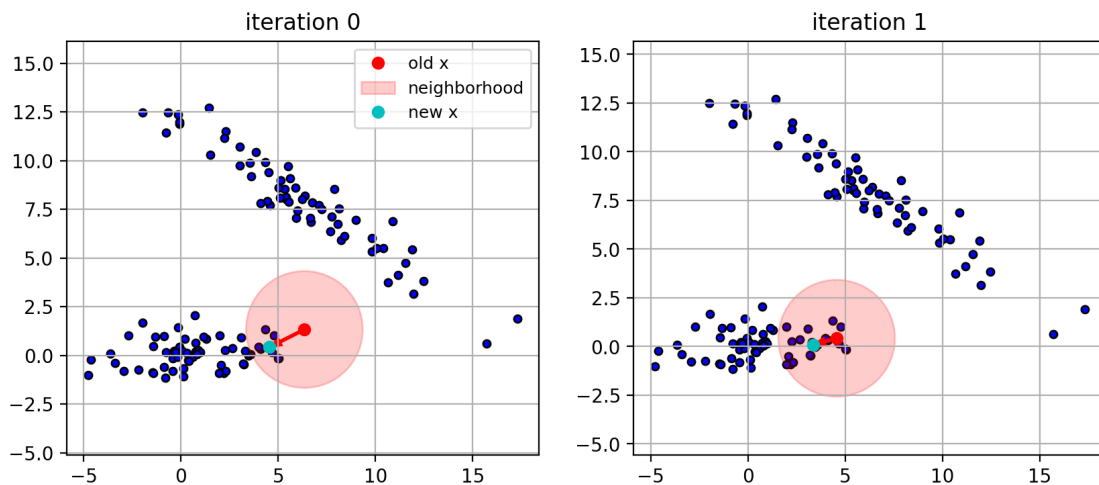
    plt.subplot(1,2,i%2+1)
    plt.scatter(X[:,0], X[:,1], c='b', edgecolors='k', s=ssize)
    plt.plot(x0[0],x0[1], 'ro', label='old x')
    circ = plt.Circle(x0, radius=3, color='r', alpha=0.2, label='neighborhood')
    plt.gca().add_patch(circ)
    plt.plot(cc[0,0],cc[0,1], 'co', label='new x')
    if (linalg.norm(x0-cc[0,:])>0.1):
        plt.arrow(x0[0], x0[1], cc[0,0]-x0[0], cc[0,1]-x0[1],
                  width=0.1, length_includes_head=True, head_width=0.4,
                  color='r')
    plt.axis('equal'); plt.grid(True)
    plt.title('iteration ' + str(i))
    if i==0:
        plt.legend(loc=0, fontsize=9)

    x0 = cc[0,:]

    if i%2 == 1:
        plt.close()
```

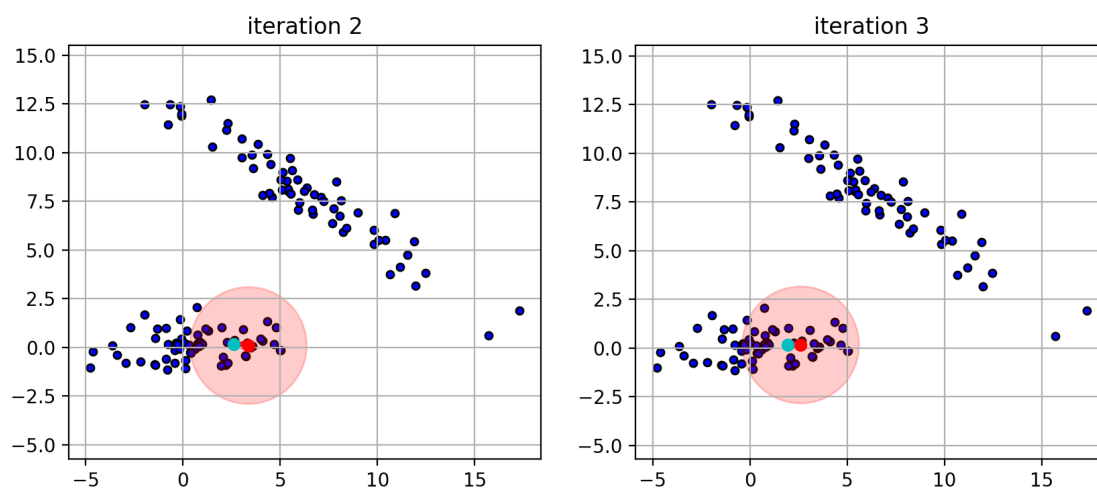
```
[11]: efigs[0]
```

```
[11]:
```



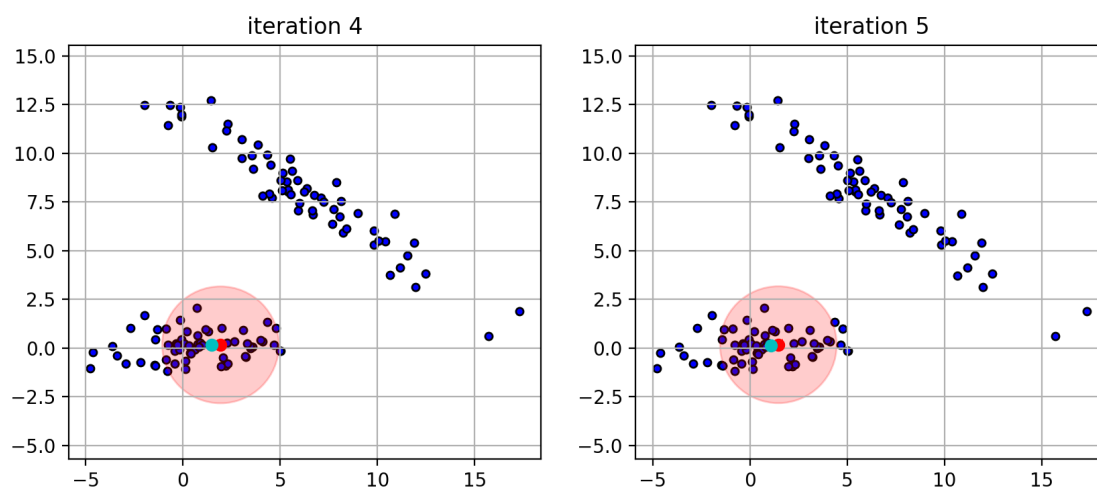
```
[12]: efigs[1]
```

```
[12]:
```



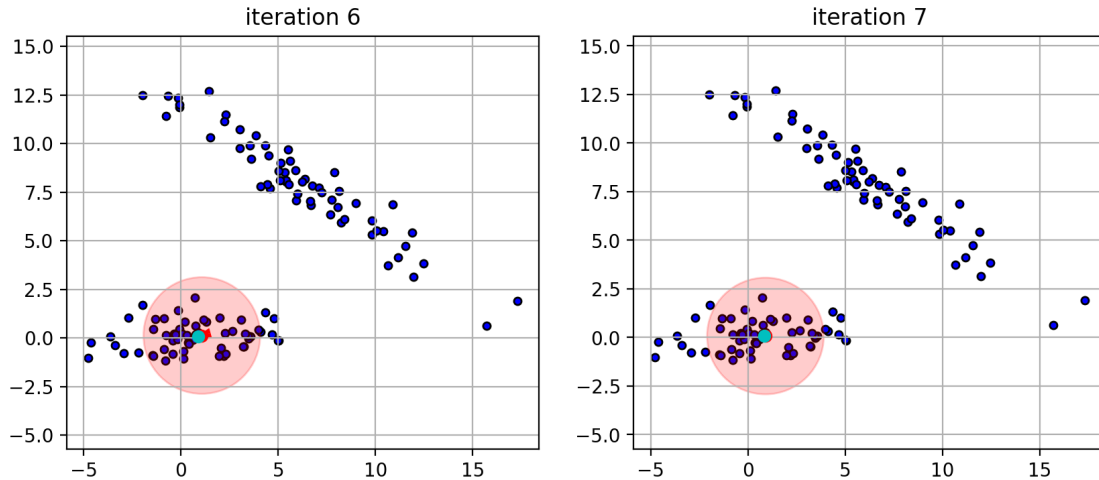
```
[13]: efigs[2]
```

```
[13]:
```



```
[14]: efigs[3]
```

```
[14]:
```

9 Getting the clusters

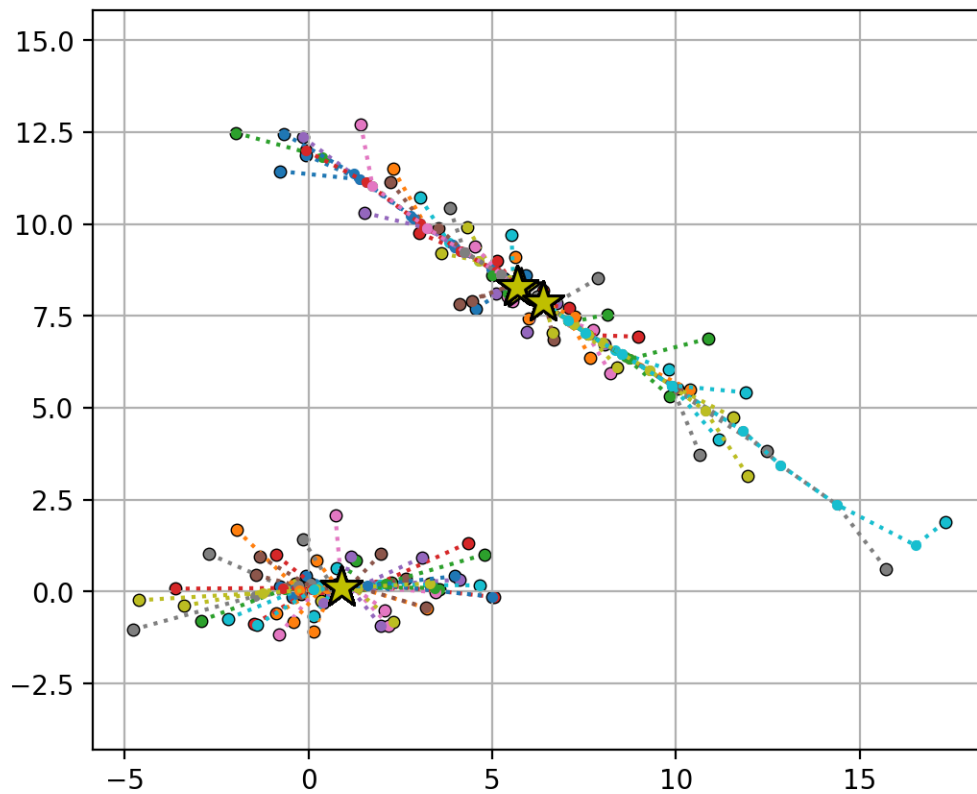
- Run the mean-shift algorithm for many initial points $\{\mathbf{x}_i\}$.
 - the set of converged points contain the cluster centers.
 - * need to remove the duplicate centers.
 - data points that converge to the same center belong to the same cluster.

```
[15]: xpath = []
      # use each point as starting point
      for x0 in Xfull:
          tmp = [x0]
          for it in range(20):
              # need to use the function version which can set the number of
              ↪ iterations
              cc, cl = cluster.mean_shift(Xfull, bandwidth=5, max_iter=0, seeds=[x0])
              tmp.append(cc[0])
              x0 = cc[0]
          xpath.append(array(tmp))
      xpath = array(xpath)
```

```
[16]: pfig = plt.figure(figsize=(6,5))
      plt.scatter(X[:,0], X[:,1], c='k', edgecolors='k', s=ssize)
      for xp in xpath:
          plt.plot(xp[:,0], xp[:,1], '.:')
      plt.scatter(xpath[:, -1, 0], xpath[:, -1, 1], s=250, marker='*', c='y', ↪
          ↪ edgecolors='k', zorder=3)
      plt.axis('equal'); plt.grid(True)
      plt.close()
```

```
[17]: pfig
```

[17]:



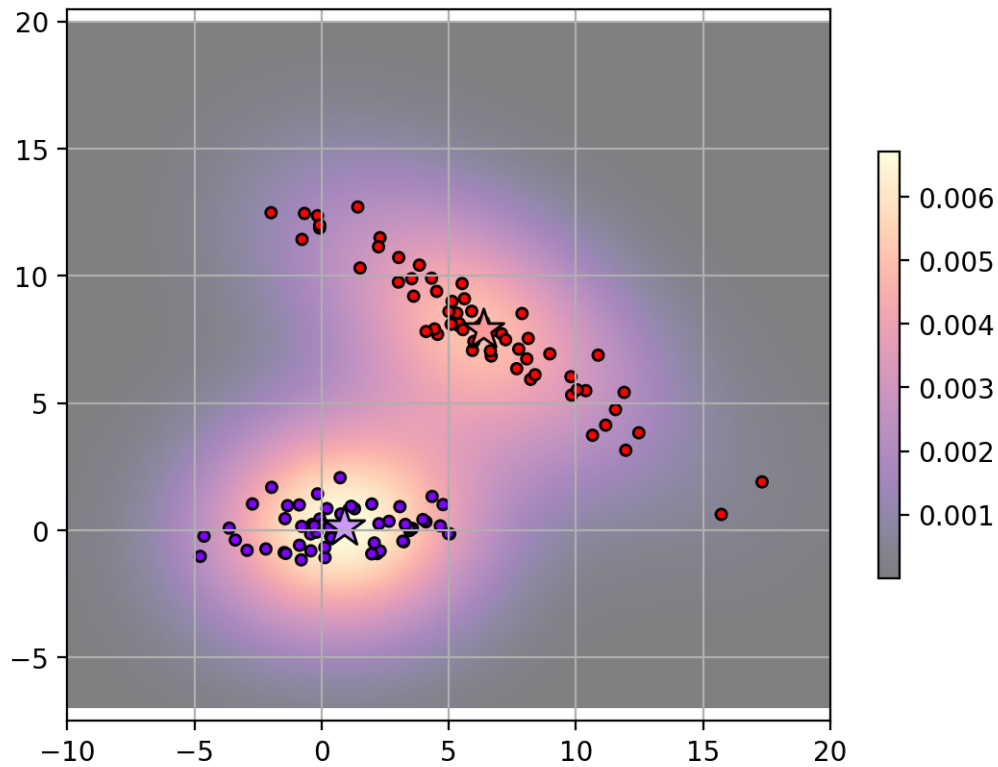
```
[18]: # bin_seeding=True -- coarsely uses data points as initial points
# different initializations can run in parallel (`n_jobs`)
ms = cluster.MeanShift(bandwidth=5, bin_seeding=True, n_jobs=-1)
Y = ms.fit_predict(X)

cc = ms.cluster_centers_ # cluster centers

plot_scores(kde, axbox, 'magma', showcontour=False)
plot_clusters(ms, axbox, X, Y, rbow, rbow2)
```

[18]: (2,)

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

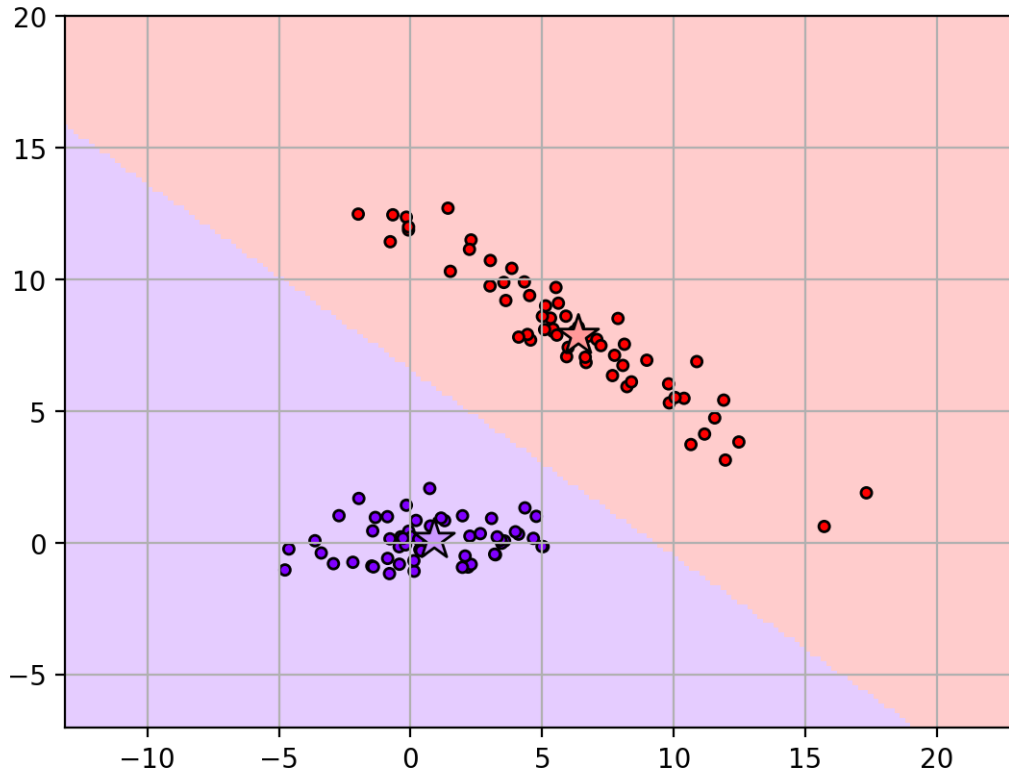


- Cluster partitions
 - assign point based on convergence to same cluster center

```
[19]: plot_clusters(ms, axbox, X, Y, rbow, rbow2, showregions=True)
```

```
[19]: (2,)
```

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.



10 Number of clusters

- Number of clusters is implicitly controlled by the bandwidth (radius of the nearest-neighbors)
 - larger bandwidth creates less clusters
 - * focuses on global large groups
 - smaller bandwidth creates more clusters
 - * focuses on local groups.

```
[20]: msfig = plt.figure(figsize=(12,5))
for i,bw in enumerate([2, 3, 5, 10]):
    ms = cluster.MeanShift(bandwidth=bw, bin_seeding=True)
    Y = ms.fit_predict(X)

    kde = neighbors.KernelDensity(bandwidth=bw).fit(X)

    plt.subplot(2,4,i+1)
    plot_scores(kde, axbox, 'magma', showcontour=True, showlabels=False)
    plt.scatter(X[:,0], X[:,1], c='b', s=ssize, edgecolors='k')
    plt.title('bandwidth={}'.format(bw))
    plt.xticks([])
    plt.yticks([])
```

```

plt.subplot(2,4,i+5)
plot_clusters(ms, axbox, X, Y, rbow, rbow2)
plt.title('{} clusters'.format(len(ms.cluster_centers_)))
plt.xticks([])
plt.yticks([])
plt.close()

```

[21]: msfig

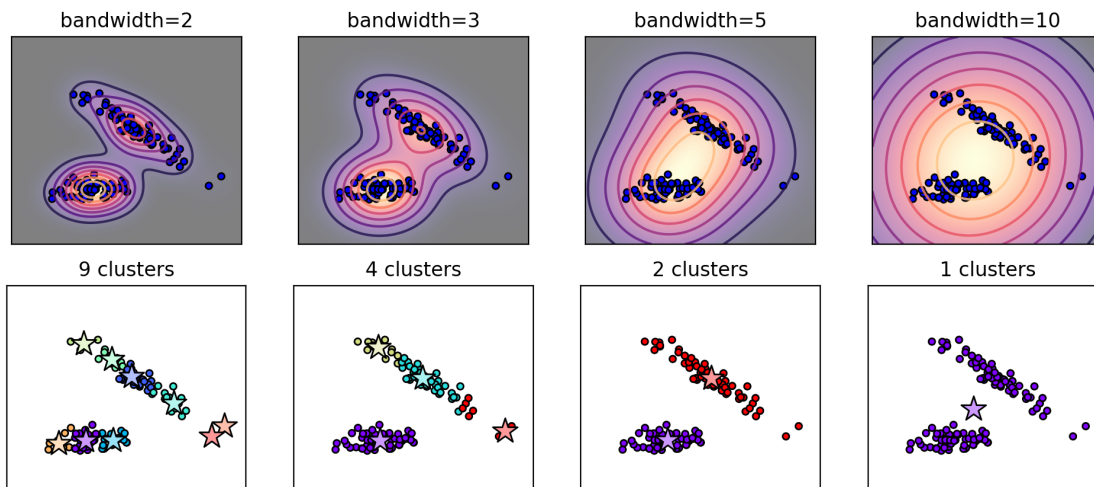
Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

[21]:



- Cluster partitions: assign points based on convergence to same cluster center.

```

[22]: msfig = plt.figure(figsize=(8,6))
for i,bw in enumerate([2, 3, 5, 10]):
    ms = cluster.MeanShift(bandwidth=bw, bin_seeding=True)
    Y = ms.fit_predict(X)

    plt.subplot(2,2,i+1)
    plot_clusters(ms, axbox, X, Y, rbow, rbow2, showregions=True)
    plt.title('bandwidth={}; {} clusters'.format(bw,len(ms.cluster_centers_)))
plt.tight_layout()
plt.close()

```

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data

limits.

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

[23]: msfig

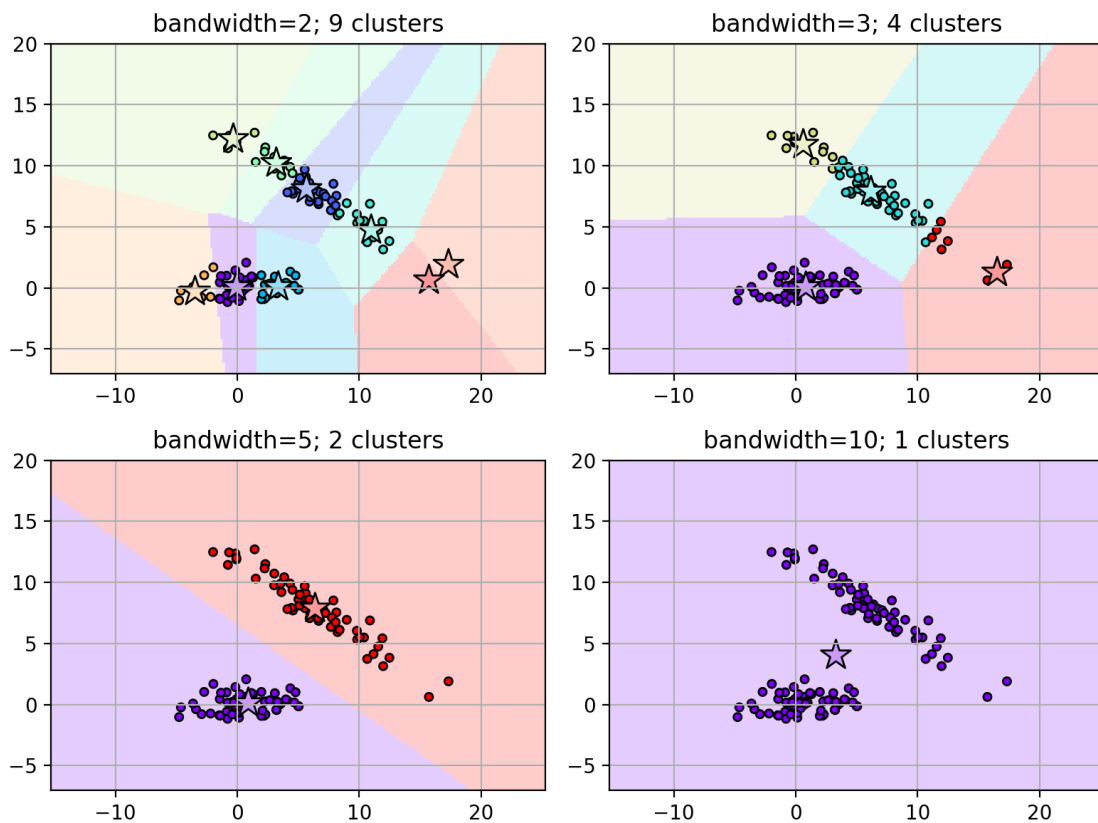
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

[23]:



11 Non-compact clusters

- K-means, GMM, and Mean-Shift assume that all clusters are compact.
 - i.e., circles or ellipses
- What about clusters of other shapes?
 - e.g., clusters not defined by compact distance to a “center”

```
[24]: random.seed(200)
X1 = 0.2*random.normal(0,1,[50, 2])
X1[:,1] *= 8
X1[:,0] += 0.3*power(X1[:,1],2) - 5

X2 = 0.4*random.normal(0,1,[50,2])-array([2,0])
X3 = 0.4*random.normal(0,1,[50,2])+array([2,0])

X4 = 0.2*random.normal(0,1,[50, 2])
X4[:,1] *= 8
X4[:,0] -= 0.3*power(X4[:,1], 2) - 5

Xtie = r_[X1,X2,X3,X4]
axboxtie = [-6, 6, -6, 6]

X = Xtie
axbox = axboxtie

km = cluster.KMeans(n_clusters=4, random_state=1234)
km.fit(X)

gmmf = mixture.GaussianMixture(n_components=4, covariance_type='full',
    ↪random_state=4487, n_init=10)
gmmf.fit(X)

ms = cluster.MeanShift(bandwidth=2, bin_seeding=True)
ms.fit(X)

tiefig = plt.figure(figsize=(10,3))
plt.subplot(1,4,1)
plt.scatter(X[:,0], X[:,1], c='b', edgecolors='k', s=ssize)
plt.axis(axbox)
plt.grid(True)
plt.title('data')

for i,m in enumerate([km, gmmf, ms]):
    plt.subplot(1,4,i+2)
    Y = m.predict(X)
    plot_clusters(m, axbox, X, Y, rbow, rbow2)
    plt.grid(True)
```

```
plt.title(m.__class__.__name__)
plt.tight_layout()
plt.close()
```

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

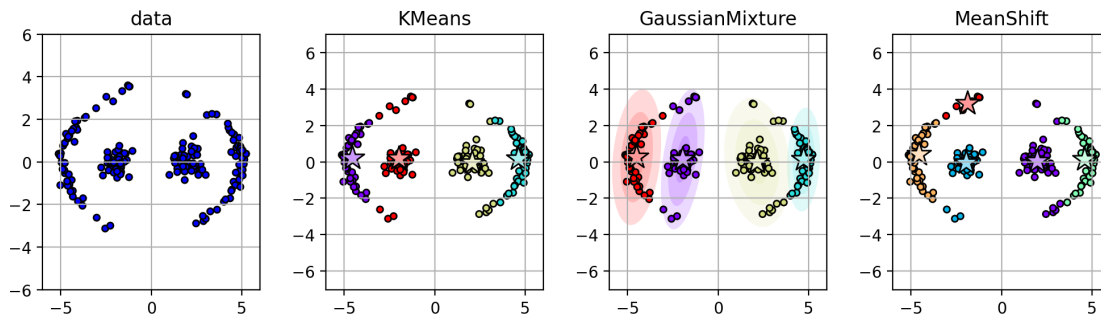
[25]: tiefig

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

[25]:



[26]: `Xcirc,Y = datasets.make_circles(n_samples=300, noise=0.05, factor=0.2)`
`axboxcirc = [-1.5, 1.5, -1.5, 1.5]`

```
X = Xcirc
axbox = axboxcirc
```

```
km = cluster.KMeans(n_clusters=2, random_state=1234)
km.fit(X)
```

```
gmmf = mixture.GaussianMixture(n_components=2, covariance_type='full',
    ↪random_state=4487, n_init=10)
gmmf.fit(X)
```

```
ms = cluster.MeanShift(bandwidth=0.5, bin_seeding=True)
ms.fit(X)
```



```

circfig = plt.figure(figsize=(8,6))
plt.subplot(2,2,1)
plt.scatter(X[:,0], X[:,1], c='b', edgecolors='k', s=ssize)
plt.grid(True); plt.axis(axbox)
plt.title('data')

for i,m in enumerate([km, gmmf, ms]):
    plt.subplot(2,2,i+2)
    Y = m.predict(X)
    plot_clusters(m, axbox, X, Y, rbow, rbow2)
    plt.grid(True)
    plt.title(m.__class__.__name__)
plt.tight_layout()
plt.close()

```

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

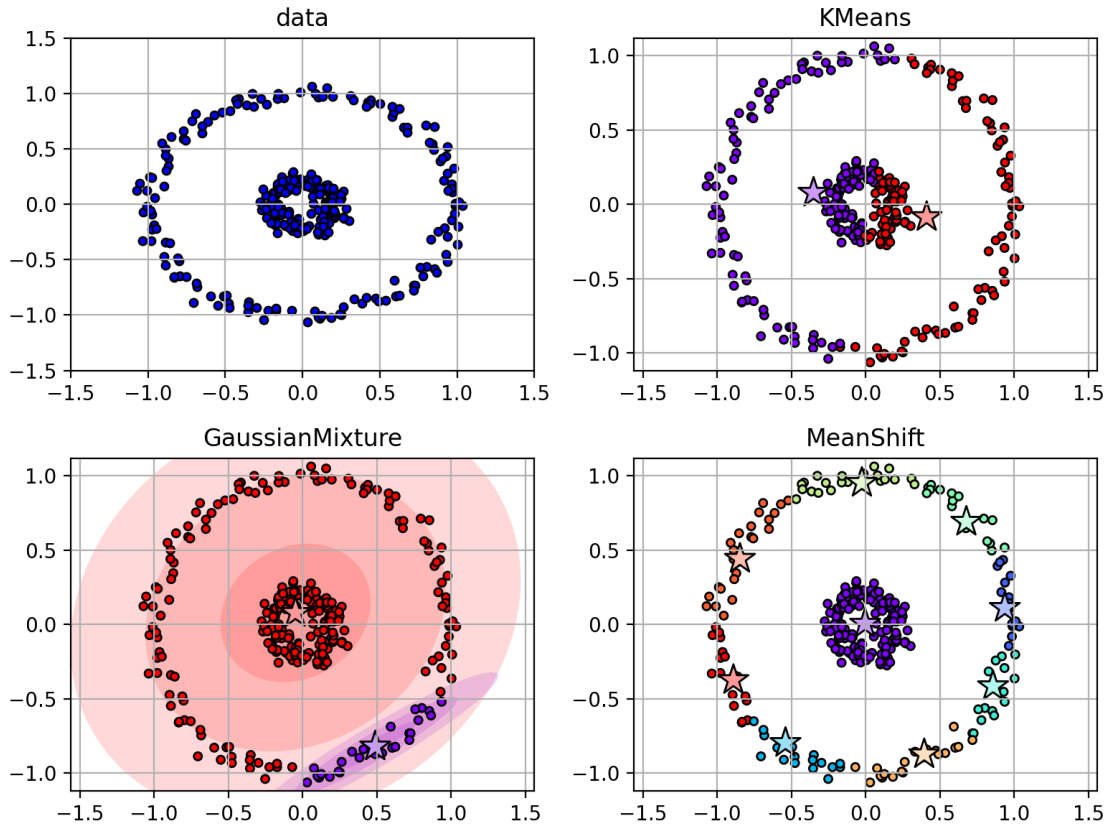
[27]: circfig

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.

[27]:



12 Spectral Clustering

- Estimate clusters using pair-wise affinity between points.
- Affinity (similarity) between points
 - kernel function: $k(\mathbf{x}_i, \mathbf{x}_j)$ – RBF kernel
 - number of nearest neighbors within a radius (bandwidth)

```
[28]: X = Xtie
axbox = axboxtie

sc = cluster.SpectralClustering(n_clusters=4, affinity='rbf', gamma=1.0,
    ↪ assign_labels='discretize', n_jobs=-1)
Y = sc.fit_predict(X) # cluster and also return the labels Y

cl = sc.labels_ # labels for each X
ii = random.permutation(len(cl))

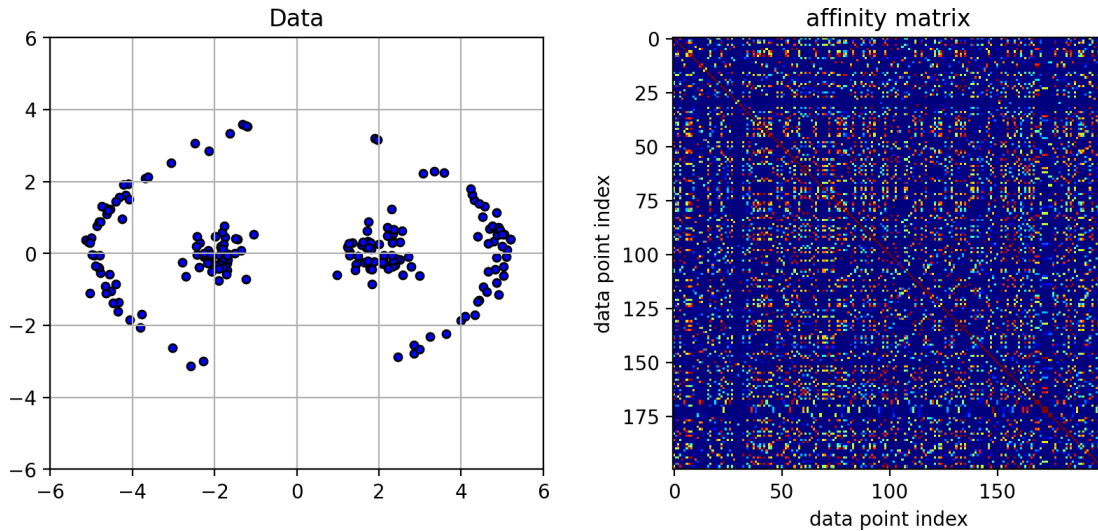
afig = plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.scatter(X[:,0], X[:,1], c='b', edgecolors='k', s=ssize)
```

```
plt.title('Data'); plt.grid(True); plt.axis(axbox)

plt.subplot(1,2,2)
plt.imshow(sc.affinity_matrix_[ix_(ii,ii)], interpolation='nearest', cmap='jet')
plt.title('affinity matrix')
plt.xlabel('data point index'); plt.ylabel('data point index')
plt.close()
```

[29]: afig

[29]:



13 Spectral Clustering

- **Idea:** clustering with a graph formulation
 - each data point is a node in a graph
 - edge weight between two nodes is the affinity $k(\mathbf{x}_i, \mathbf{x}_j)$
 - * (darker colors indicate stronger weights)

```
[30]: def plot_sc_graph(X, gamma, axbox):
    Xlen = X.shape[0]
    # calculate kernel
    K = metrics.pairwise.rbf_kernel(X, gamma=gamma)

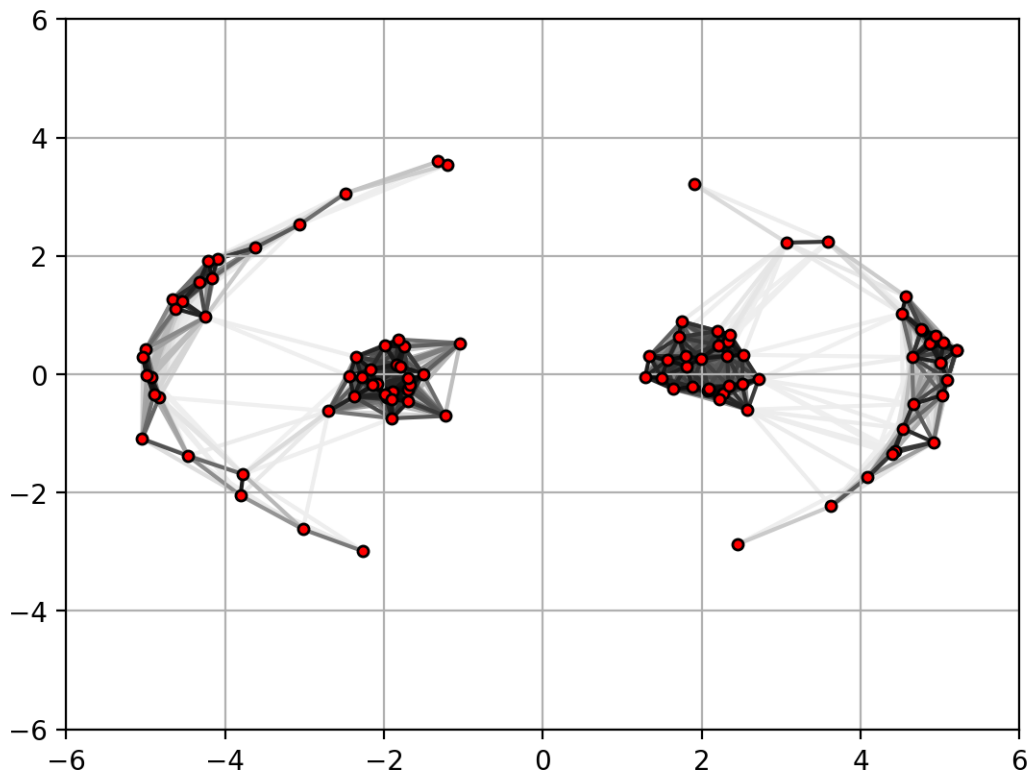
    # graph figure
    for i in range(Xlen):
        for j in range(i+1, Xlen):
            if (K[i,j] > 0.01):
                plt.plot([X[i,0], X[j,0]], [X[i,1], X[j,1]],
                        color=0.95*(1-K[i,j])*ones(3), zorder=K[i,j])
```

```
# plot points
plt.scatter(X[:,0], X[:,1], zorder=2, c='r', edgecolors='k', s=ssize)
plt.axis(axbox); plt.grid(True)
```

```
X = Xtie[:,2,:]
graphfig = plt.figure()
plot_sc_graph(X, 1.0, axbox)
plt.close()
```

```
[31]: graphfig
```

```
[31]:
```



- **Goal:** cut the graph into clusters such that weights of cut edges is small compared to the total edge weight within each cluster.
 - find “blocks” of high affinity in the affinity matrix.
- Intuitively, consider a “mass-spring” system – masses connected together with springs.
 - the graph nodes are masses, the edge weights are the spring stiffness.
 - if you hit the masses...
 - * the masses that are tightly connected by stiff springs will move together in a low-frequency vibration mode.
- These low-frequency modes are found with the smallest non-zero eigenvectors of the graph

Laplacian

- Graph Laplacian: $\mathbf{L} = \mathbf{D} - \mathbf{A}$
 - $\mathbf{A}$ is the adjacency (affinity) matrix.
 - $\mathbf{D}$ is the degree matrix.
 - * diagonal matrix with entry $D_{ii} = \sum_j A_{i,j}$
- There are different ways to define the Laplacian, leading to different versions of Spectral Clustering
 - the one in sklearn is called “Normalized Cuts”.

```
[32]: X = Xtie
      axbox = axboxtie
```

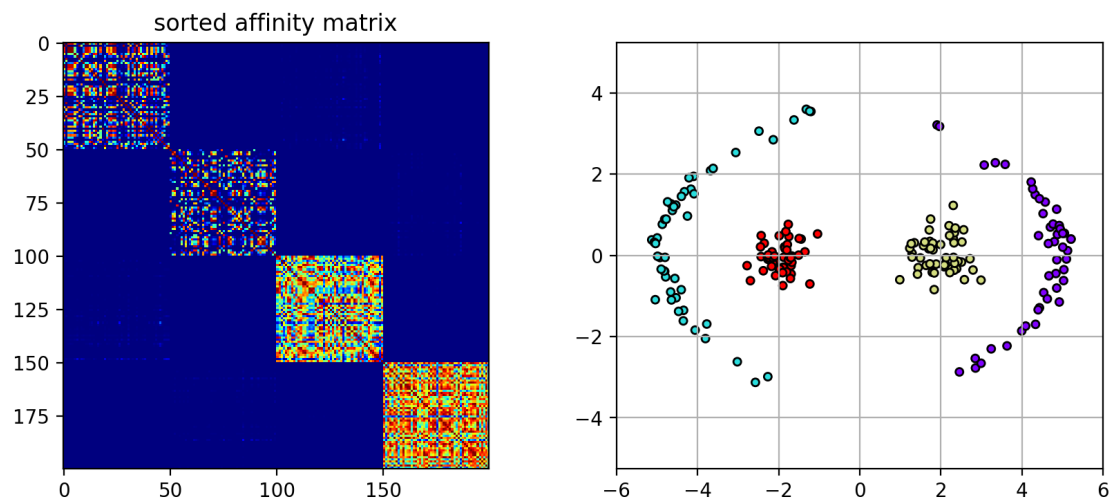
```
[33]: # spectral clustering
      # rbf affinity
      sc = cluster.SpectralClustering(n_clusters=4, affinity='rbf',
                                     gamma=1.0, assign_labels='discretize',
                                     ↪n_jobs=-1)
      Y = sc.fit_predict(X)
```

```
[34]: ii = argsort(Y)
      scfig = plt.figure(figsize=(10,4))
      plt.subplot(1,2,1)
      plt.imshow(sc.affinity_matrix_[ix_(ii,ii)], interpolation='nearest', cmap='jet')
      plt.title('sorted affinity matrix')

      plt.subplot(1,2,2)
      plot_clusters(sc, axbox, X, Y, rbow, rbow2)
      plt.close()
```

```
[40]: scfig
```

```
[40]:
```



14 Sensitivity to gamma

- gamma controls which structures are important
 - small gamma - far away points are still considered similar
 - large gamma - only close points are considered as similar

```
[41]: X = Xtie
axbox = axboxtie

scfig2 = plt.figure(figsize=(12,5))
for i,g in enumerate([1.0, 10.0, 20., 100]):
    sc = cluster.SpectralClustering(n_clusters=4, affinity='rbf', gamma=g,
                                   assign_labels='discretize', n_jobs=-1)

    Y = sc.fit_predict(X)

    plt.subplot(2,4,i+1)
    ii = argsort(Y)
    plt.imshow(sc.affinity_matrix_[ix_(ii,ii)], interpolation='nearest',
               cmap='jet')
    plt.title('gamma=' + str(g))
    plt.xticks([]); plt.yticks([])
    plt.subplot(2,4,i+5)
    plot_clusters(sc, axbox, X, Y, rbow, rbow2)
    plt.xticks([]); plt.yticks([])

plt.close()
```

```
[42]: scfig2
```

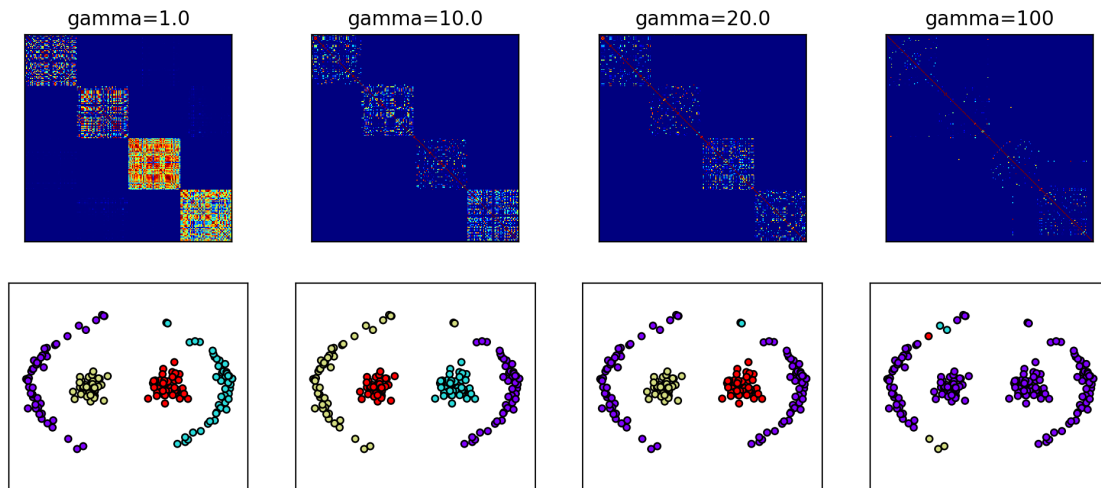
Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

```
[42]:
```

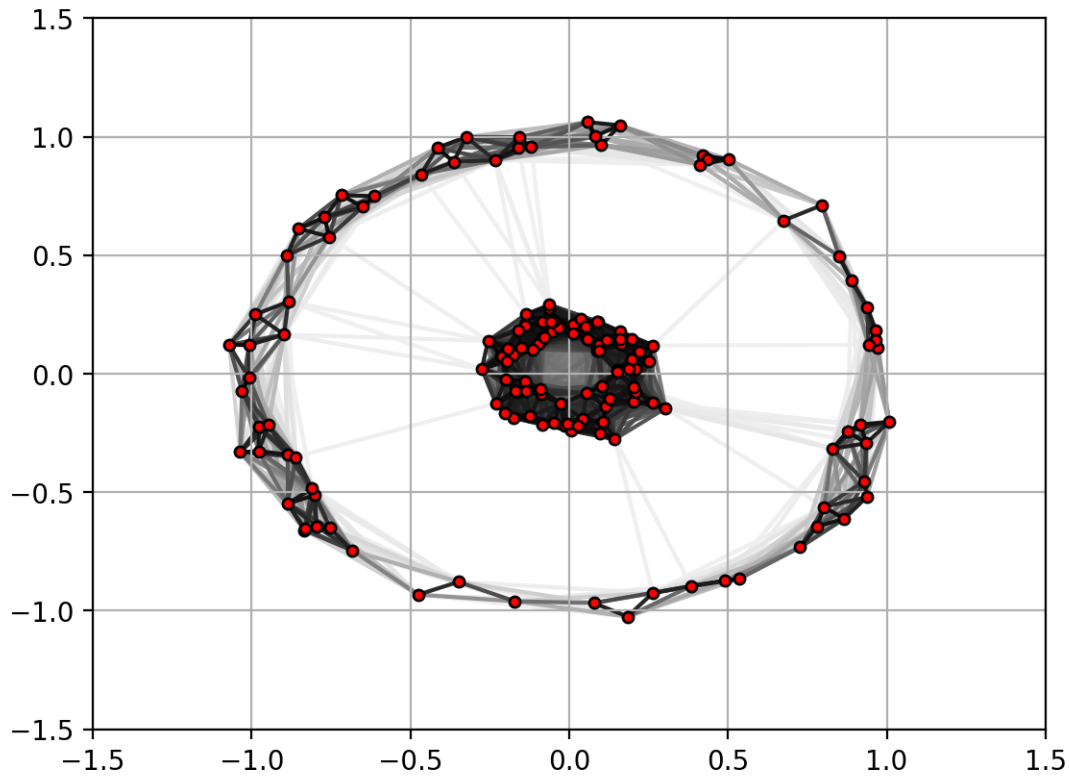


15 Another Example

```
[43]: X = Xcirc[:,2,:]
      axbox = axboxcirc
      graphfig2 = plt.figure()
      plot_sc_graph(X, 10.0, axbox)
      plt.close()
```

```
[44]: graphfig2
```

```
[44]:
```



```
[45]: X = Xcirc
axbox = axboxcirc
sc = cluster.SpectralClustering(n_clusters=2, affinity='rbf',
                                gamma=10, assign_labels='discretize', n_jobs=-1)
Y = sc.fit_predict(X)

ii = argsort(Y)

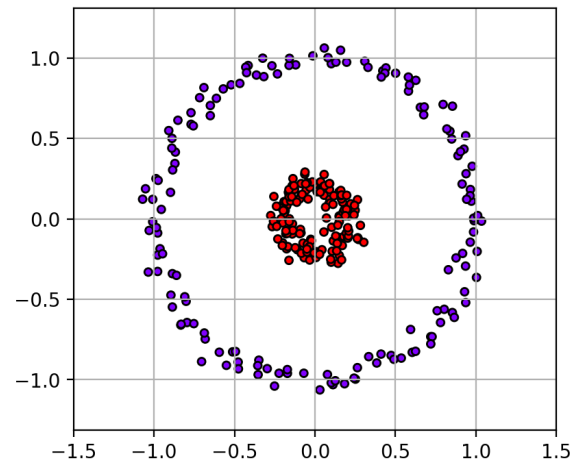
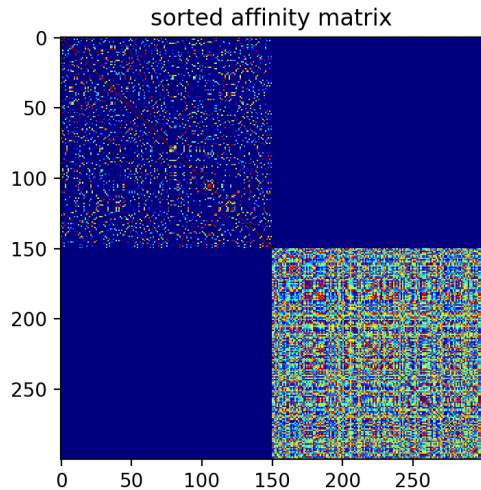
scfig = plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.imshow(sc.affinity_matrix_[ix_(ii,ii)], interpolation='nearest', cmap='jet')
plt.title('sorted affinity matrix')

plt.subplot(1,2,2)
plot_clusters(sc, axbox, X, Y, rbow, rbow2)
plt.close()
```

```
[46]: scfig
```

Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

```
[46]:
```

16 Clustering Summary

- **Goal:** given set of input vectors $\{\mathbf{x}_i\}_{i=1}^n$, with $\mathbf{x}_i \in \mathbb{R}^d$, group similar x_i together into clusters.
 - estimate a cluster center, which represents the data points in that cluster.
 - predict the cluster for a new data point.

17 Other Things

- *Feature normalization*
 - feature normalization is typically required clustering.
 - e.g., algorithms based on Euclidean distance (Kmeans, Mean-Shift, Spectral Clustering)

18 Example

- scaling down the x_1 feature makes its differences less important, compared to x_2 .

```
[47]: random.seed(38)
Xbars,Y = datasets.
↪make_blobs(400,2,centers=[[-1,-1],[-1,1],[1,1],[1,-1]],cluster_std=0.8/5)

allXbars = [Xbars]
allXbars.append( Xbars * array([0.2, 1.]) )
allXbars.append( Xbars * array([0.1, 1.]) )

efig = plt.figure(figsize=(10,4.5))

for i,X in enumerate(allXbars):
    km = cluster.KMeans(n_clusters=4, random_state=4487)
    Yp = km.fit_predict(X)
```

```

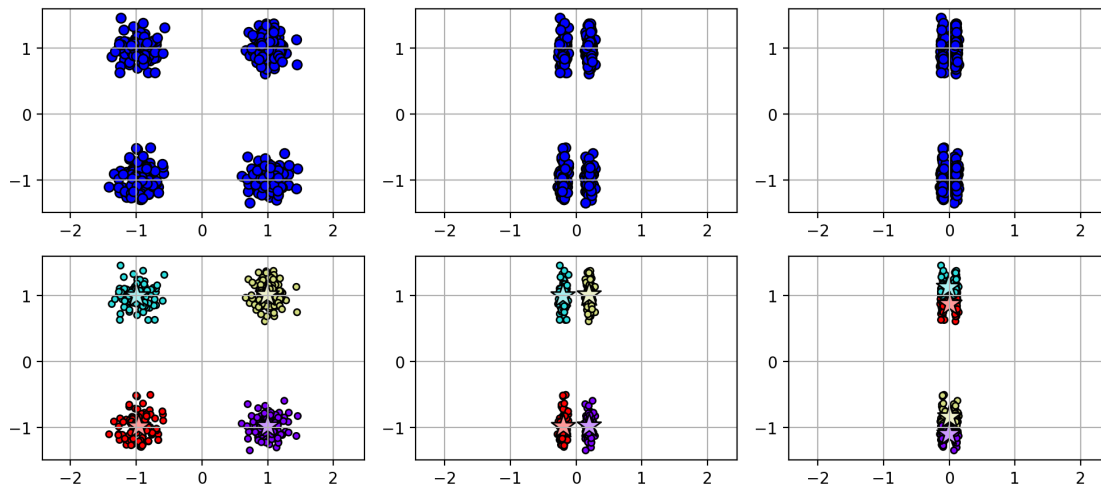
plt.subplot(2,3,1+i)
plt.scatter(X[:,0], X[:,1], c='b', edgecolors='k')
plt.axis('equal'); plt.grid(True)

plt.subplot(2,3,4+i)
plot_clusters(km, axbox, X, Yp, rbow, rbow2)
plt.axis('equal'); plt.grid(True)
plt.tight_layout()
plt.close()

```

[48]: efig

[48]:



[]: